

Highlights

Certified evidence contracts for reliable and cost-efficient retrieval-augmented generation over heterogeneous knowledge sources

Kang Yao, Congcong Tian, Quanbo Cheng, Shengjiang Zhang, Hao Yin, Jinjiang Cui, Weiwei Fu

- Evidence contracts declare auditable reliability targets for RAG knowledge services
- Fixed-sequence tests certify cost-efficient policies with finite-sample guarantees
- Assumption-free risk vouchers order a lattice of cost-decreasing plan rewrites
- Anytime-valid drift monitor stays valid over repeated alarms and subsampled audits
- Population-risk guarantees hold across four LLM families at up to 20× lower cost

Certified evidence contracts for reliable and cost-efficient retrieval-augmented generation over heterogeneous knowledge sources

Kang Yao^{a,b}, Congcong Tian^c, Quanbo Cheng^{a,b}, Shengjiang Zhang^{a,b}, Hao Yin^{a,b}, Jinjiang Cui^{b,*} and Weiwei Fu^{b,**}

^aSchool of Biomedical Engineering (Suzhou), Division of Life Sciences and Medicine, University of Science and Technology of China, Suzhou, China

^bSuzhou Institute of Biomedical Engineering and Technology, Chinese Academy of Sciences, Suzhou, China

^cFaculty of Computing, Harbin Institute of Technology, Harbin, China

ARTICLE INFO

Keywords:

Retrieval-augmented generation
Evidence contracts
Distribution-free risk control
Large language models
Knowledge drift
Certified deployment

ABSTRACT

Retrieval-augmented generation (RAG) grounds large language models (LLMs) in external knowledge sources, yet the reliability of the resulting knowledge service remains uncalibrated: practitioners tune pipelines on development sets and hope the observed quality persists in deployment. Existing adaptive RAG systems reduce cost with routing or cascading heuristics, but none can state, with quantified confidence, what fraction of deployed answers will violate an application's quality, citation, freshness, or latency requirements. We propose *evidence contracts*, a declarative specification of admissible violation rates for a RAG knowledge service, and CONTRACTRAG, a framework that certifies contract feasibility by statistical testing rather than point-estimate comparison. CONTRACTRAG organizes cost-decreasing pipeline rewrites into a plan lattice annotated with assumption-free *risk vouchers*, materializes progressive knowledge-acquisition policies — including policies that defer to human review under strict contracts — over the lattice, and deploys the cheapest policy in the prefix certified by a distribution-free fixed-sequence test on calibration data, yielding a finite-sample guarantee that all contract constraints hold with probability at least $1 - \delta$. After deployment, an anytime-valid e-process monitor — valid also under subsampled auditing — detects knowledge drift with a change-time-independent delay bound, localizes the changepoint, and drives a recalibration loop whose geometric error-budget schedule preserves validity over arbitrarily many alarms. Experiments on four heterogeneous benchmarks (HybridQA, CRAG, ASQA, QAMPARI) with fully metered execution verify the guarantee directly on exact finite-population risk: across 1000 repeated calibration draws, CONTRACTRAG's deployed policies exceed their contract in at most 0.8% of draws against a 10% budget, versus up to 64% for Bayesian optimization and empirical tuning, while cutting LLM cost by up to 20× against the strongest fixed pipeline (10–18× after charging monitoring audits). The guarantee transfers unchanged across four LLM families from four model developers and two serving backends, including a locally served open-weight family metered in GPU-seconds. These results suggest that knowledge-intensive LLM systems should move beyond point-estimate tuning toward certified, contract-driven deployment.

1. Introduction

Large language models (LLMs) are increasingly deployed as knowledge-intensive components of production software systems, powering question answering over enterprise documents, regulatory and financial assistants, and clinical decision support (Lewis, Perez, Piktus, Petroni, Karpukhin, Goyal, Küttler, Lewis, tau Yih, Rocktäschel, Riedel and Kiela, 2020; Gao, Yen, Yu and Chen, 2023a; Yang, Sun, Xin, Sun, Bhalla, Chen, Choudhary, Gui, Jiang, Jiang et al., 2024; Aarab, 2025; Yang, Some, Bain and Kang, 2025). Because parametric knowledge is incomplete, stale, and prone to hallucination, retrieval-augmented generation (RAG) has become the default mechanism for grounding LLM outputs in external knowledge sources — relational tables, text corpora under sparse and dense indexes, vector stores, and structured knowledge APIs (Lewis et al., 2020;

Chen, Zha, Chen, Xiong, Wang and Wang, 2020). A modern RAG service is therefore a *knowledge acquisition pipeline*: between retrieval and generation sit fusion, reranking, verification, and a menu of LLMs whose per-token prices span two orders of magnitude. The same information need admits many pipeline configurations, and their cost differences are large; their quality differences are real but stochastic. For knowledge services that face compliance, safety, or service-level obligations, a property more fundamental than average accuracy is at stake: *can the operator state, before violations reach users, what fraction of answers will fail the application's quality, citation, freshness, or latency requirements?*

However, the dominant deployment practice cannot answer this question. Practitioners sample a development set, estimate each configuration's quality, and deploy the cheapest configuration whose *estimate* clears the target — a paradigm we call point-estimate tuning (Figure 1a), shared by workload-level pipeline optimizers (Russo, Sudhir, Vitagliano, Liu, Kraska, Madden and Cafarella, 2025; Liu, Russo, Cafarella, Cao, Chen, Chen, Franklin, Kraska, Madden and Vitagliano, 2025; Patel, Jha, Guestrin and Zaharia, 2024) and by adaptive RAG routers and LLM cascades (Jeong, Baek, Cho, Hwang and Park, 2024; Chen,

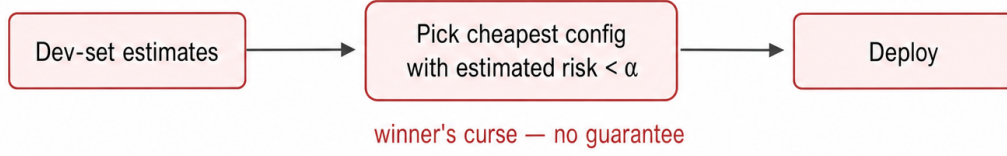
*Corresponding author

**Corresponding author

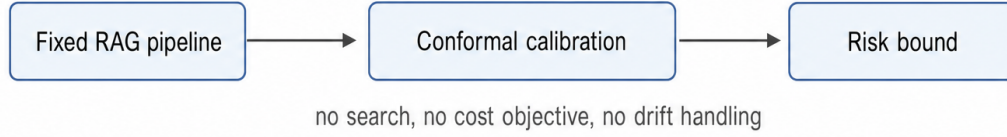
✉ yaokang@mail.ustc.edu.cn (K. Yao); 24b903119@stu.hit.edu.cn (C. Tian); qbcheng@ustc.edu.cn (Q. Cheng); zhangshengjiang@mail.ustc.edu.cn (S. Zhang); hao_yin@mail.ustc.edu.cn (H. Yin); cuijj@sibet.ac.cn (J. Cui); fuww@sibet.ac.cn (W. Fu)

ORCID(s):

(a) Point-estimate tuning



(b) Risk control on a fixed pipeline



(c) ContractRAG (ours)

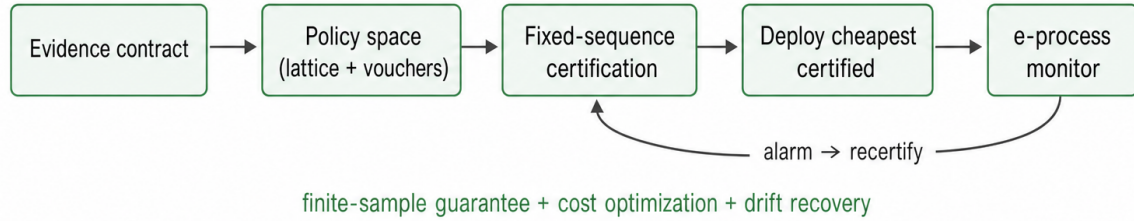


Figure 1: Three deployment paradigms for RAG knowledge services. (a) Point-estimate tuning deploys the cheapest configuration whose *measured* risk clears the target — an uncorrected multiple-testing procedure with no feasibility guarantee. (b) Distribution-free risk control bounds the risk of a *fixed* pipeline but neither searches a configuration space nor reduces cost, and is silent under drift. (c) CONTRACTRAG unifies search, certification, and monitoring: an evidence contract is certified by a fixed-sequence test over a policy space ordered by risk vouchers, and an anytime-valid e-process monitor closes the loop after deployment.

Zaharia and Zou, 2023; Ong, Almahairi, Wu, Chiang, Wu, Gonzalez, Kadous and Stoica, 2024). This paradigm suffers from a fundamental statistical flaw: selecting the cheapest configuration whose measured risk clears a threshold is a multiple-testing procedure without a correction, so the winner is systematically the configuration whose noise broke in its favor. The failure is structural rather than a matter of tuning budget — more candidates make it worse, not better. As we quantify in Section 6, state-of-practice optimizers deploy policies whose true violation rate exceeds the declared target in up to 64% of calibration draws: for a compliance requirement, a coin flip. Moreover, even a correctly tuned pipeline offers no protection after deployment, when workload mix shifts, indexes fall behind the world, and knowledge APIs degrade — the knowledge drift problem that static evaluation cannot see.

Distribution-free risk control offers a principled remedy (Figure 1b): conformal and Learn-then-Test methods (Angelopoulos, Bates, Candès, Jordan and Lei, 2025; Angelopoulos, Bates, Fisch, Lei and Schuster, 2024) can bound the violation rate of a *fixed* pipeline with finite-sample confidence, and recent work has applied them to RAG abstention and answer sets (Li, Park, Lee and Bastani, 2024; Kang, Gürel, Yu, Song and Li, 2024). Yet these methods treat the pipeline as given: they neither search a configuration space nor reduce cost, and they are silent about what happens when the calibrated distribution drifts. Conversely, the systems

that do search — routers, cascades, Bayesian optimization — provide no feasibility statement at all. What is missing is a framework in which *search*, *certification*, and *post-deployment monitoring* are designed together, so that cost optimization never erodes the statistical guarantee.

To close this gap, we propose CONTRACTRAG, a certified deployment framework for RAG knowledge services (Figure 1c). The key idea is to make the reliability promise a first-class, declarative object: an *evidence contract* states admissible violation rates for answer quality, citation completeness, evidence freshness, and latency, together with a confidence budget δ for the certification itself. CONTRACTRAG then treats contract feasibility the way a database treats integrity constraints: no pipeline configuration — however attractive its point estimate — is deployable unless a statistical test on calibration executions certifies it. Three mechanisms make this practical. First, cost-decreasing pipeline rewrites (retrieval truncation, reranker elision, pre-filter pushdown, generator downgrade) are annotated with empirical *risk vouchers* — confidence upper bounds on their probability of harming an answer — which compose assumption-free along rewrite chains by a telescoping argument and guide the search the way a cost model guides a query optimizer: they order candidates but never justify the deployment. Second, the rewrite lattice is lifted into *progressive knowledge-acquisition policies* — escalation ladders that execute cheap plans first and escalate while runtime

sufficiency evidence is inadequate, cost-utility routers, and group-conditional variants, and, under strict contracts, policies that defer residual queries to human review with a certified deferral budget — and a fixed-sequence Hoeffding–Bentkus test certifies a feasible low-cost policy (the cheapest of its certified prefix) with no multiplicity penalty, so enriching the policy space never erodes the guarantee. Third, an anytime-valid restart-mixture e-detector watches deployed violations — at full or subsampled audit rates — with a change-time-independent detection-delay bound, localizes the changepoint at $O(1)$ cost per query, and drives a closed-loop certified recalibration procedure whose geometric error-budget schedule remains valid over arbitrarily many alarms. This design elevates RAG deployment from point-estimate tuning to certified knowledge-service management: learned components decide how much money the certified policy saves, while statistics alone decides whether the contract holds.

To the best of our knowledge, this is among the first frameworks to provide end-to-end, finite-sample reliability guarantees for cost-optimized RAG over heterogeneous knowledge sources. The main contributions of this paper are summarized as follows:

- We propose *evidence contracts*, a declarative and statistically meaningful interface between knowledge-intensive applications and the RAG execution layer, subsuming quality, citation, freshness, and latency requirements (Section 3).
- We design a certified plan optimizer that combines rewrite-generated plan lattices, assumption-free telescoping voucher composition, selectivity-conditioned vouchers, and cardinality-driven knowledge-source selection, with fixed-sequence testing as the sole source of soundness; we prove contract validity for any candidate count and any score quality, and a cost-consistency guarantee (Section 4).
- We develop an anytime-valid drift monitor based on a restart-mixture e-detector, with a proven change-time-independent detection delay bound, $O(1)$ changepoint localization, validity under subsampled auditing, a certified post-alarm recalibration loop, and a lifetime guarantee across repeated alarms via geometric error spending (Section 4.6).
- We conduct a systematic empirical study on four heterogeneous benchmarks with fully metered execution across four LLM families from four model developers on two serving backends (token- and GPU-second-metered). Verified directly on exact finite-population risk, CONTRACTRAG is the only method meeting its contract budget everywhere (violation probability at most 0.8% over 1000 draws against $\delta = 10\%$); it cuts cost by up to 20 $\times$ against the strongest fixed pipeline (10–18 $\times$ after charging monitoring audits), certifies strict contracts down to $\alpha = 0.1$ with deferral and

four-constraint joint contracts where Bonferroni certifies nothing, detects injected drift in a median of 78 queries with no false alarms, and reports infeasibility honestly when no configuration can meet the contract (Sections 5–6).

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 formalizes evidence contracts and the certified deployment problem. Section 4 presents the CONTRACTRAG framework. Sections 5 and 6 report the experimental setup and results. Section 7 discusses implications, limitations, and threats to validity, and Section 8 concludes.

2. Related work

CONTRACTRAG sits at the intersection of retrieval-augmented generation, adaptive LLM inference, distribution-free risk control, and runtime monitoring of knowledge systems. We review each line and position our work against it.

2.1. Retrieval-augmented generation and knowledge-enhanced LLMs

RAG grounds LLM outputs in retrieved evidence to mitigate hallucination and knowledge staleness (Lewis et al., 2020; Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun, Wang and Wang, 2023b; Yang et al., 2025). A rich line of work improves the knowledge acquisition process itself: hybrid retrieval over tables and text (Chen et al., 2020), self-reflective retrieval decisions (Asai, Wu, Wang, Sil and Hajishirzi, 2024), active and corrective retrieval during generation (Jiang, Xu, Gao, Sun, Liu, Dwivedi-Yu, Yang, Callan and Neubig, 2023; Yan, Gu, Zhu and Ling, 2024), evidence compression (Xu, Shi and Choi, 2024), cost-conscious QA system design under resource constraints (Budur, Özçelik, Soylu, Khattab, Güngör and Potts, 2024), calibration between parametric and retrieved knowledge (Deng, Shi, Wu, Huo, Wang and Xu, 2026), and benchmark suites that stress heterogeneous and time-sensitive knowledge sources (Yang et al., 2024; Stelmakh, Luan, Dhingra and Chang, 2022; Amouyal, Wolfson, Rubin, Yoran, Herzig and Berant, 2023; Gao et al., 2023a; Chen, Lin, Han and Sun, 2024). Automated evaluation frameworks quantify RAG quality post hoc (Es, James, Espinosa-Anke and Schockaert, 2024; Saad-Falcon, Khattab, Potts and Zaharia, 2024; Ru, Qiu, Hu, Zhang, Shi, Chang, Jiayang, Wang, Sun, Li, Zhang, Wang, Jiang, He, Wang, Liu, Zhang and Zhang, 2024), typically via LLM judges (Zheng, Chiang, Sheng, Zhuang, Wu, Zhuang, Lin, Li, Li, Xing, Zhang, Gonzalez and Stoica, 2023), but do not constrain the deployed system’s behavior. Declarative pipeline frameworks — Palimpsest (Liu et al., 2025), LOTUS (Patel et al., 2024), DocETL (Shankar, Parameswaran and Wu, 2024), DSPy (Khattab, Singhvi, Maheshwari, Zhang, Santhanam, Vardhamanan, Haq, Sharma, Joshi, Moazam, Miller, Zaharia and Potts, 2024), and the Abacus optimizer (Russo et al., 2025) — compile semantic operator pipelines and search spaces of models, prompts,

and operator implementations against validation estimates. These systems optimize average quality or cost but compare point estimates to constraints; none provides a finite-sample statement about the deployed pipeline’s true violation rate. In contrast, CONTRACTRAG retains the pipeline search perspective but replaces estimate-feasibility with statistical certification, and static pipelines with certified adaptive policies.

2.2. Adaptive inference: LLM routing and cascades

Cost-aware inference systems route queries to the cheapest adequate model or cascade through model tiers: Fru-galGPT (Chen et al., 2023), RouteLLM (Ong et al., 2024), Hybrid-LLM (Ding, Mallick, Wang, Sim, Mukherjee, Rühle, Lakshmanan and Awadallah, 2024), AutoMix (Aggarwal, Madaan, Anand, Potharaju, Mishra, Zhou, Gupta, Rajagopal, Kappaganthu, Yang, Upadhyay, Faruqi and Mausam, 2024), and Adaptive-RAG’s complexity-based routing (Jeong et al., 2024), with RouterBench (Hu, Bieker, Li, Jiang, Keigwin, Ranganath, Keutzer and Upadhyay, 2024) standardizing their evaluation. Confident adaptive computation (Schuster, Fisch, Gupta, Dehghani, Bahri, Tran, Tay and Metzler, 2022) and conformal cascades (Fisch, Schuster, Jaakkola and Barzilay, 2021) calibrate early exits for a single model or a cascade with shared scores. These methods reduce cost effectively, but their selection rules are tuned empirically, so the realized violation rate is whatever it happens to be — our experiments measure 16–64% contract violations across calibration draws for representative members of this family. Unlike prior routing work, CONTRACTRAG subsumes routing as one candidate family within a certified policy space and supplies the missing feasibility guarantee; unlike single-model early exits, it calibrates heterogeneous multi-source pipelines under multi-dimensional contracts.

2.3. Distribution-free risk control for language models

Conformal prediction and risk control give finite-sample guarantees without distributional assumptions (Angelopoulos et al., 2024). Learn-then-Test (Angelopoulos et al., 2025) casts hyperparameter selection as multiple testing with Hoeffding–Bentkus p-values (Bentkus, 2004); TRAQ (Li et al., 2024) builds conformal sets over retrieved passages and answers; C-RAG (Kang et al., 2024) certifies generation risk for a given pipeline; conformal language modeling and conformal factuality calibrate stopping and abstention rules on fixed models (Quach, Fisch, Schuster, Yala, Sohn, Jaakkola and Barzilay, 2024; Mohri and Hashimoto, 2024), with recent extensions to conditional validity and alignment filtering (Cherian, Gibbs and Candès, 2024; Gui, Jin and Ren, 2024); semantic-entropy methods detect hallucinations without gold labels (Farquhar, Kossen, Kuhn and Gal, 2024). A complementary line stress-tests LLM reliability offline, e.g., by logic-grounded metamorphic testing (Zhou, Li, Fang, Zhou, Lin and Zheng, 2026); such audits expose defects but issue no quantitative guarantee about a deployed

configuration. These works establish that RAG risk *can* be controlled distribution-free, but they treat the pipeline as given: no plan search, no cost objective, no adaptive escalation, and no drift handling. CONTRACTRAG employs the same statistical machinery but embeds it inside an optimizer, where the tested objects are adaptive policies over a rewrite lattice and the objective is monetary cost; the certification discipline is chosen precisely so that its validity survives unlimited search.

2.4. Runtime monitoring and drift adaptation in knowledge systems

Deployed knowledge systems drift: workload mix shifts, indexes fall behind the world, and external APIs degrade. Anytime-valid inference — test supermartingales and e-processes (Ville, 1939; Shafer, 2021; Ramdas, Grünwald, Vovk and Shafer, 2023; Grünwald, de Heide and Koolen, 2024) — provides sequential tests with always-valid error control, and e-detectors (Shin, Ramdas and Rinaldo, 2024) extend them to changepoint detection. Database systems faced an analogous problem in mid-query re-optimization (Kabra and DeWitt, 1998; Markl, Raman, Simmen, Lohman, Pirahesh and Cilimdžić, 2004), re-planning when runtime observations diverge from optimizer estimates. CONTRACTRAG transplants this discipline to knowledge services: a restart-mixture e-detector with a proven change-time-independent delay bound watches audited violations, an $O(1)$ recursion localizes the changepoint, and a certified recalibration loop restores the contract on the post-change distribution. To our knowledge, no prior RAG or LLM-cascade system closes this loop with statistical guarantees at every stage.

3. Preliminaries and problem formulation

This section formalizes hybrid RAG pipelines as plans over a knowledge operator algebra, introduces evidence contracts, and defines the certified deployment problem.

3.1. Hybrid RAG plans over heterogeneous knowledge sources

Let S be a set of evidence sources: relational tables, text corpora with sparse and dense indexes, and remote structured knowledge APIs. A *plan* P is a directed acyclic graph built from typed operators

$\text{Retrieve}_{s,k} \mid \text{Fuse} \mid \text{Rerank}_m \mid \text{Filter}_\phi \mid \text{Generate}_g \mid \text{Repair}$,

where $\text{Retrieve}_{s,k}$ fetches k evidence units from source s (SQL row selection, BM25, dense approximate nearest neighbor, or API calls), Fuse merges rankings (e.g., reciprocal-rank fusion), Rerank_m re-scores with a cross-encoder m , Filter_ϕ drops evidence failing a predicate ϕ (e.g., an entailment or freshness test), Generate_g produces the answer with LLM g , and Repair post-edits an answer using verification feedback. Executing P on a query q yields an answer, an evidence set E with a citation map, and a realized execution vector (cost, latency, evidence ages, ...) metered

at run time. Two syntactically different RAG plans are almost never output-equivalent; the notion of equivalence must be statistical, which motivates the formulation below.

3.2. Evidence contracts

Definition 1 (Evidence contract). Let $\mathcal{D}$ be a query distribution. An evidence contract is a tuple $C = \langle (\ell_j, \alpha_j)_{j=1}^k, \delta \rangle$ where each $\ell_j : (\pi, q) \mapsto [0, 1]$ is a bounded violation loss — for example, $\ell_{\text{qual}} = \mathbf{1}[\text{quality}(\pi(q)) < \tau_q]$, $\ell_{\text{cite}} = \mathbf{1}[\text{citation recall}(\pi(q)) < \tau_c]$, $\ell_{\text{fresh}} = \mathbf{1}[\max_{e \in E(q)} \text{age}(e) > \Delta_f]$, $\ell_{\text{lat}} = \mathbf{1}[\text{latency}(\pi(q)) > B_\ell]$ — $\alpha_j \in (0, 1)$ are admissible violation rates, and $\delta \in (0, 1)$ bounds the probability that the deployed policy fails to satisfy the contract at all.

As a running example, consider a financial-services assistant whose compliance office imposes requirements on the service, not on any single answer: at most 10% of answers may be incorrect, at most 10% may contain a claim without an entailed citation or cite evidence older than 72 hours, and at most 5% may exceed a 10-second latency budget — all at minimal LLM spend. These requirements instantiate Definition 1 directly, with four losses, $\alpha = (.10, .10, .10, .05)$, and $\delta = 0.1$: with 90% confidence over the calibration draw, the deployed pipeline violates none of the four rate limits.

A policy π maps a query to an execution schedule over plans, possibly running several plans adaptively (Section 4.4). Its risks are $R_j(\pi) = \mathbb{E}_{q \sim \mathcal{D}}[\ell_j(\pi, q)]$ and its cost is $\text{cost}(\pi) = \mathbb{E}_{q \sim \mathcal{D}}[c(\pi, q)]$ with c the metered monetary cost. The optimization problem is

$$\min_{\pi \in \Pi} \text{cost}(\pi) \quad \text{s.t.} \quad R_j(\pi) \leq \alpha_j \quad \forall j, \quad (1)$$

where Π is a policy space induced by the plan algebra. The distribution $\mathcal{D}$ is unknown; the optimizer sees n independent calibration queries with realized losses. Contract satisfaction can therefore only ever be established statistically.

Definition 2 ((C, δ)-certified deployment). A data-dependent selection procedure $\mathcal{A}$ mapping a calibration sample $Z_{1:n}$ to a policy $\hat{\pi}$ is (C, δ) -certified if

$$\mathbb{P}_{Z_{1:n}}(R_j(\hat{\pi}) \leq \alpha_j \quad \forall j) \geq 1 - \delta.$$

This is a qualitatively different guarantee from what existing RAG optimizers provide. Workload-level optimizers such as Abacus (Russo et al., 2025) compare point estimates of plan quality against the constraint; adaptive routers (Jeong et al., 2024) provide no feasibility statement at all. Both can — and, as we show in Section 6, do — deploy policies whose true violation rate exceeds α in a large fraction of calibration draws. Conformal RAG methods (Li et al., 2024; Kang et al., 2024) do control risk, but for a single fixed pipeline; they leave the cost-minimization dimension of (1) untouched.

Remark 1 (What the guarantee is about). A contract is always stated with respect to a loss functional the operator can audit: the guarantee of Definition 2 is exact for whatever ℓ_j is plugged in — token-F1 against gold answers, NLI-based citation entailment, an LLM judge following a

benchmark’s official protocol, evidence age, or wall-clock latency. The system enforces the declared predicate exactly and does not adjudicate whether the predicate captures the application’s intent. If the operator upgrades the auditor — for example, replacing the LLM judge with periodic human audits on the calibration sample — every statement in this paper carries over unchanged, because the machinery only requires bounded, independent losses, never that the loss be machine-computed. What the certificate does *not* promise is per-query correctness or robustness to a systematically biased auditor; Section 7 discusses this limitation.

Two features of (1) shape the framework design. First, the policy space Π must be searched, not enumerated: it contains per-query adaptive policies, not just static plans. Second, the certificate must survive *selection*: testing many candidates on the same calibration data and picking a passing one invalidates naive confidence intervals. CONTRACTRAG is built around a fixed-sequence multiple-testing discipline that pays no union-bound penalty for searching an arbitrarily long candidate sequence (Section 4.5).

4. The CONTRACTRAG framework

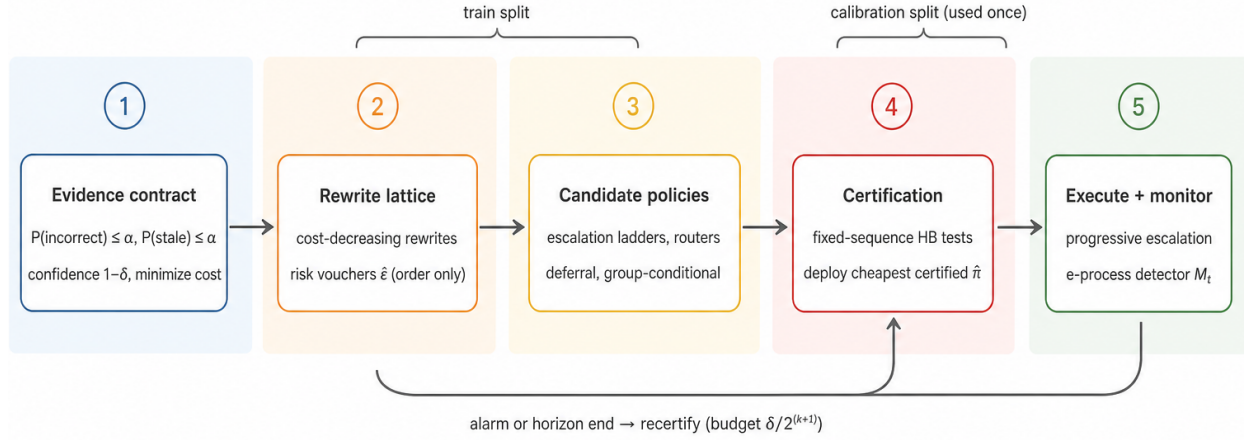
4.1. Framework overview

Figure 2 shows the architecture. CONTRACTRAG consists of four components. (i) A *rewrite layer* generates a lattice of cost-decreasing plan variants, each annotated with an empirical *risk voucher* that upper-bounds its probability of harming an answer. (ii) A *policy layer* lifts static plans into per-query adaptive policies: escalation ladders, cost-utility routers, and group-conditional variants. (iii) A *certification layer* walks the candidate policies in a data-independent order and outputs the cheapest member of the prefix certified by a distribution-free risk test. (iv) A *monitoring layer* watches deployed violations with an anytime-valid e-detector and triggers certified recalibration on alarm. Only the certification layer touches calibration data, and it does so exactly once; the rewrite and policy layers use a disjoint training split. Vouchers and learned scores guide the search order — like a cost model in a database optimizer, they may be wrong without compromising correctness, only optimality. This division of labor is the central design principle: *machine learning decides how much the certified policy saves; statistics alone decides whether the contract holds.*

4.2. Cost-decreasing rewrites and risk vouchers

A rewrite rule $R : P \mapsto P'$ transforms a plan into a cheaper one (Figure 3): retrieval truncation ($k \rightarrow k' < k$), reranker elision, source pre-filter pushdown (restricting a passage pool to rows selected by a structured predicate), generator downgrade ($g \rightarrow g'$ with a cheaper model), and repair elision. Classical query optimizers require $P \equiv P'$; here equivalence fails, and we quantify the damage.

Definition 3 (Risk voucher). For loss ℓ and rewrite R , let $\rho(R; P) = \mathbb{P}_q(\ell(R(P), q) > \ell(P, q))$ be the harm probability. An $(\hat{\epsilon}, \delta_r)$ -voucher for R at P is a Clopper–Pearson



learned components order the search; statistics alone decides deployability

Figure 2: Overview of the CONTRACTRAG framework. An evidence contract (stage 1) declares the reliability targets; a rewrite layer (stage 2) generates a lattice of cost-decreasing plan variants annotated with risk vouchers (Section 4.2); a policy layer (stage 3) lifts static plans into progressive knowledge-acquisition policies (Section 4.4); a certification layer (stage 4) walks the candidates in a train-data order and deploys the cheapest policy in the certified prefix of a distribution-free fixed-sequence risk test (Section 4.5); the executor runs the certified policy while an anytime-valid e-process monitor (stage 5) defends the certificate against knowledge drift, re-entering certification on an alarm or at the end of a monitoring horizon with geometrically scheduled budgets (Section 4.6). Stages 2–3 use the training split only; calibration data is touched exactly once, in stage 4.

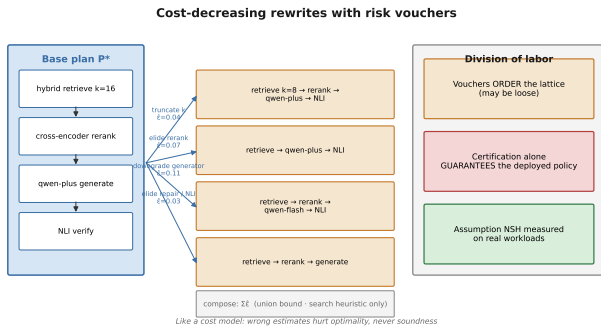


Figure 3: The rewrite lattice. Each edge is a cost-decreasing rewrite annotated with a risk voucher; a maximal chain of rewrites yields the cost-ascending ladder used by the policy layer.

upper bound $\hat{e}$ on $\rho(R; P)$ at confidence $1 - \delta_r$, computed from training executions of P and $R(P)$ on the same queries.

Assumption-free composition along paths The naive way to compose vouchers measures each rewrite against the base plan and sums the bounds; that requires a no-synergistic-harm (NSH) assumption — that the chain harms no query unless some individual rewrite already does — which real workloads violate (Section 6.3). CONTRACTRAG instead vouchers the *path*: for the chain $P_0 \xrightarrow{R_1} P_1 \xrightarrow{R_2} \dots \xrightarrow{R_m} P_m$, each step voucher $\hat{e}_i^{\text{step}}$ bounds the harm of R_i against its actual predecessor P_{i-1} , not against P_0 .

Proposition 1 (Telescoping path composition). *If $\ell(P_m, q) > \ell(P_0, q)$ then some consecutive step strictly increases the loss, so deterministically the event $\{q : \ell(P_m, q) > \ell(P_0, q)\}$ is contained in $\bigcup_i \{q : \ell(P_i, q) > \ell(P_{i-1}, q)\}$. Hence if each step carries an $(\hat{e}_i^{\text{step}}, \delta_r)$ -voucher, then with probability at least $1 - m\delta_r$, $\rho(R_m \dots R_1; P_0) \leq \sum_{i=1}^m \hat{e}_i^{\text{step}}$ — with no behavioral assumption on rewrite interactions.*

The inclusion is a tautology of the telescoping decomposition $\ell(P_m) - \ell(P_0) = \sum_i [\ell(P_i) - \ell(P_{i-1})]$; the only stochastic step is the m Clopper–Pearson bounds (proof in A). The price is that step vouchers are chain-specific, which matches how the optimizer explores anyway: it descends one path of the rewrite lattice at a time. *Selectivity-conditioned vouchers* refine this further: the harm of a pre-filter push-down concentrates on queries whose structured predicate is unselective (the restricted pool drops relevant evidence), so CONTRACTRAG stratifies calibration queries by estimated pool selectivity σ (Section 4.3) and issues a per-stratum Clopper–Pearson bound with a Bonferroni correction across strata. The result is a voucher that is tight where the rewrite is safe and honest where it is not. Even so, CONTRACTRAG never uses vouchers as deployment guarantees: they order the search space, while soundness comes exclusively from direct certification of the deployed policy.

4.3. Cardinality-driven knowledge-source selection

Hybrid RAG queries *join* a structured source with an unstructured one: on HybridQA-style workloads, the evidence

must connect table rows (selected by a structured predicate) with linked passages. Two acquisition orders exist. *Table-first* pushes the predicate down — select rows, then retrieve only from the selected rows’ passage pool (small pool, cheap, but drops evidence linked from unselected rows); *text-first* retrieves over the whole table’s pool and joins back to rows (large pool, higher recall on vague predicates). Which order wins is decided by the predicate’s *selectivity*. CONTRACTRAG estimates cardinalities before retrieval: the number of rows matching the predicate, and the pushdown and full pool sizes from the link table, giving a pool selectivity $\sigma = |T_{\text{push}}|/|T_{\text{full}}|$ per query. A cost model fit by least squares on measured executions and a recall model $r(\sigma)$ fit on the training split drive a classical selection rule: the cheapest acquisition path whose estimated recall clears the target; otherwise the highest-recall path. As with vouchers, cardinality estimates affect only cost, never the certificate. The same estimates index the selectivity-conditioned vouchers of Section 4.2, so the planner and the risk layer share one selectivity abstraction.

Operator sharing and materialization Retrieval operators are generator-independent, so CONTRACTRAG materializes each (query, rung) retrieval result once and shares it across every generator family and every policy evaluated downstream. Certification itself never re-executes plans: all candidates are evaluated against the materialized execution matrix, which is what makes plan spaces of 10^5 candidates tractable (Section 6.6).

4.4. Progressive knowledge-acquisition policies

From the rewrite lattice, CONTRACTRAG materializes a *cost-ascending ladder* $P_0 < P_1 < \dots < P_{L-1}$ — a maximal chain, for example from sparse-only retrieval with a flash-tier model up to hybrid retrieval with reranking, verification, and a max-tier model. Each execution of rung j on query q emits a *sufficiency score* $s_j(q) \in [0, 1]$: a calibrated probability that stopping here satisfies the contract, predicted from runtime observables only — retrieval margins, cross-encoder scores, NLI support of the drafted answer over its evidence, citation coverage, evidence ages, and abstention markers. Scores are fit on the training split by per-rung logistic models.

Three candidate families lift the ladder into policies:

- **T-family (escalation)**: thresholds $\lambda = (\lambda_0, \dots, \lambda_{L-2})$; execute rungs upward and stop at the first $s_j(q) \geq \lambda_j$ (the final rung always answers). Thresholds are parameterized along a one-dimensional quantile path from most conservative to always-stop.
- **U-family (routing)**: jump directly to the rung maximizing $\hat{s}_j(q) - \lambda \bar{c}_j$ over j — a utility router sweeping λ over a grid; it pays only the chosen rung’s cost but forfeits the escalation fallback.
- **F-family (static)**: the L fixed plans themselves.
- **D-family (selective escalation)**: any of the above with one more knob κ — if even the final rung’s

sufficiency score falls below κ , the query is *deferred* to human review instead of answered. Deferral turns the contract into the pair $\mathbb{P}(\text{answered} \wedge \text{violating}) \leq \alpha$ and $\mathbb{P}(\text{deferred}) \leq \beta$; both losses are bounded, so the same certification machinery applies unchanged. This family is what makes strict contracts ($\alpha \leq 0.2$) deployable on workloads where every always-answer plan is infeasible (Section 6.2).

Group-conditional variants (per query type or per dynamism class) apply any family within each group. For a grouping $G(q) \in \{1..m\}$ known at query time, CONTRACTRAG certifies a threshold family per group on the group’s calibration subsample, each at level δ/m , yielding the stronger guarantee $R_j(\hat{\pi} \mid G = g) \leq \alpha_j$ for all groups simultaneously with probability at least $1 - \delta$. The candidate set $C = \{\pi_1, \pi_2, \dots\}$ is finite but large; every candidate is fully specified by training data alone.

The escalation family realizes, in statistical form, the database idea of progressive re-optimization (Kabra and DeWitt, 1998; Markl et al., 2004): classical progressive execution compares runtime cardinalities against optimizer estimates and re-plans when they diverge; CONTRACTRAG compares runtime sufficiency evidence against certified thresholds and escalates to a stronger plan when the current rung cannot demonstrate contract compliance. The critical difference is that the escalation rule itself carries the contract guarantee (Theorem 1): the thresholds are not tuned heuristics but the output of risk-controlled selection, so “how sure must rung j be before answering” is exactly as conservative as the contract demands, no more.

4.5. Certification by fixed-sequence testing

For candidate π and constraint j , the empirical risk on n calibration queries is $\hat{R}_j(\pi)$. We use the Hoeffding–Bentkus p-value (Bentkus, 2004; Angelopoulos et al., 2025) for the null hypothesis $H_0^{(j)}(\pi) : R_j(\pi) > \alpha_j$:

$$p_{\pi}^{(j)} = \min \left(e^{-n h_1(\hat{R}_j \wedge \alpha_j)}, e \mathbb{P}(\text{Bin}(n, \alpha_j) \leq \lceil n \hat{R}_j \rceil) \right),$$

with h_1 the Bernoulli Kullback–Leibler divergence. For multi-constraint contracts the null is a union $\bigcup_j H_0^{(j)}$, so by the intersection–union principle (Berger, 1982) the test “reject iff $p_{\pi}^{(j)} \leq \delta$ for every j ” has size at most δ — no Bonferroni splitting across constraints is needed.

Candidates are ordered $\pi_{(1)}, \pi_{(2)}, \dots$ by training-split risk (safest first), ties broken by training cost — an ordering fixed before calibration data is seen. CONTRACTRAG walks the sequence, certifies each candidate whose composite test rejects, and stops at the first failure; among certified candidates it deploys the cheapest by calibration cost. Algorithm 1 summarizes the complete deployment cycle, including the post-deployment monitoring loop of Section 4.6. Two budget-accounting regimes must not be conflated. A *single-epoch* deployment — certify once, no monitoring loop — may spend the whole budget on its one certification; this is the setting of Theorem 1 and of the certification experiments in Section 6. *Lifetime* operation instead treats the

Algorithm 1 Certified lifetime deployment of a RAG knowledge service

Require: contract $C = \{(\mathcal{E}_j, \alpha_j)\}_{j \leq d}$ with *lifetime* budget δ ; train split Z_{tr} ; calibration split $Z_{1:n}$; plan lattice $\mathcal{P}$

- 1: annotate each rewrite edge of $\mathcal{P}$ with a risk voucher $\hat{\epsilon}_r$ on Z_{tr} ; compose vouchers along chains by telescoping (Proposition 1)
- 2: build candidate policies Π (ladders, routers, deferral, group-conditional) on Z_{tr} ; fix the walk order $\pi_{(1)}, \pi_{(2)}, \dots$ by training risk, vouchers breaking ties
- 3: **for** epochs $k = 0, 1, 2, \dots$ with budget $\delta_k = \delta/2^{k+1}$ **do**
- 4: $Z^{(k)} \leftarrow$ epoch- k calibration data ($Z_{1:n}$ for $k = 0$; the fresh post-change window for $k \geq 1$)
- 5: $S \leftarrow \emptyset$ {certified prefix}
- 6: **for** $i = 1, 2, \dots, |\Pi|$ **do**
- 7: compute $p_{\pi_{(i)}}^{(j)}$ on $Z^{(k)}$ for every constraint j
- 8: **if** $\max_j p_{\pi_{(i)}}^{(j)} \leq \delta_k/2$ **then**
- 9: $S \leftarrow S \cup \{\pi_{(i)}\}$
- 10: **else**
- 11: **break** {first failure stops the walk}
- 12: **end if**
- 13: **end for**
- 14: **if** $S = \emptyset$ **then**
- 15: report *infeasible*; stay in flagged safe mode {never silently deployed}
- 16: **else**
- 17: deploy $\hat{\pi}_k \leftarrow \arg \min_{\pi \in S} \hat{c}(\pi)$
- 18: **end if**
- 19: run a fresh horizon- T monitor M_t of Eq. (2) at level $\delta_k/2$ on audited violations
- 20: **if** $M_t \geq 2/\delta_k$ **or** horizon T exhausted **then**
- 21: enter safe mode; collect a fresh post-change calibration window; advance to epoch $k + 1$
- 22: **end if**
- 23: **end for**

initial certification as epoch 0's certification: Algorithm 1 runs every epoch's walk at level $\delta_k/2$ — the initial one at $\delta_0/2 = \delta/4$, not δ — so certifications and monitors over *all* epochs sum to exactly $\sum_{k \geq 0} \delta_k = \delta$ (Proposition 5); no stage borrows error probability from the future.

Theorem 1 (Contract validity). *Let $\hat{\pi}$ be the policy returned by the fixed-sequence procedure. Then $\mathbb{P}(R_j(\hat{\pi}) \leq \alpha_j \forall j) \geq 1 - \delta$, i.e., CONTRACTRAG is (C, δ) -certified, for any number of candidates, any score quality, and any distribution $\mathcal{D}$.*

The proof (A) rests on the walk stopping at the first non-rejection: an infeasible candidate can be certified only if the walk reaches it and its size- δ test falsely rejects. Two properties matter in practice. *Selection-proofness*: the guarantee is unaffected by how many candidates precede the stopping point, so enriching Π (new routers, new ladders) never costs statistical validity. *Power*: if the training-risk order is aligned with the true order — which we verify

empirically — the walk penetrates deep into the cheap region before stopping.

Because the walk stops at its first failure, what the procedure returns is, by construction, *the cheapest policy of the certified prefix* — not necessarily the cheapest feasible policy of the whole family. The following proposition states when the two coincide and at what statistical price; when its ordering premise fails, validity (Theorem 1) is untouched and only optimality degrades.

Proposition 2 (Cost consistency). *Let M be the total number of candidates. Suppose along the walk order the true risks are non-decreasing up to the cheapest feasible candidate $\pi^\dagger$, which sits at position $K \leq M$ with margin $R(\pi^\dagger) \leq \alpha - \gamma$, $\gamma > 0$, and that per-query costs lie in $[0, C_{\max}]$. Then with probability at least $1 - 2\delta'$ the procedure certifies every candidate up to and including $\pi^\dagger$ once $n \gtrsim \gamma^{-2}(\log(1/\delta) \vee \log(K/\delta'))$, and the deployed policy $\hat{\pi}$ — selected as cheapest by estimated calibration cost among all certified candidates, which may extend beyond position K — satisfies*

$$c(\hat{\pi}) \leq c(\pi^\dagger) + 2C_{\max} \sqrt{\frac{\log(2M/\delta')}{2n}},$$

where $c(\cdot)$ is the true mean cost: the uniform cost-estimation error is controlled over the full candidate set — since the certified set is not known in advance — and vanishes at rate $O(\sqrt{\log M/n})$.

The $\log K$ and $\log M$ terms are the price of certifying the K -prefix and estimating all M costs simultaneously; both enter only logarithmically, so enriching the candidate set is statistically cheap. (Alternatively, restricting the cheapest-selection to the first K walk positions, or estimating costs on an independent split, replaces $\log M$ by $\log K$ at no cost to validity.) Certification cost is negligible computationally: it re-uses the ladder executions already collected for calibration and needs no additional LLM calls beyond the $n \times L$ calibration matrix.

4.6. Anytime-valid drift monitoring and certified recalibration

Certificates are only as good as the exchangeability between calibration and deployment; deployed knowledge services drift — workload mix shifts, indexes fall behind the world, APIs degrade. CONTRACTRAG monitors deployed violation indicators $\ell_t \in [0, 1]$ with a *restart-mixture* e-detector (Shin et al., 2024; Ramdas et al., 2023) (Figure 4). Fix a horizon T and a finite bet grid $W \subset (0, 1/\alpha)$. For restart time r and bet w , the wealth process is $E_t^{(r,w)} = \prod_{i=r}^t (1 + w(\ell_i - \alpha))$ (with $E_t^{(r,w)} = 1$ for $t < r$). Rather than taking a maximum over restarts — which is not an e-process and cannot be scaled by Ville's inequality — the detector statistic is the weighted mixture

$$M_t = \sum_{r=1}^t \sum_{w \in W} \frac{1}{T|W|} E_t^{(r,w)} + \left(1 - \frac{t}{T}\right), \quad (2)$$

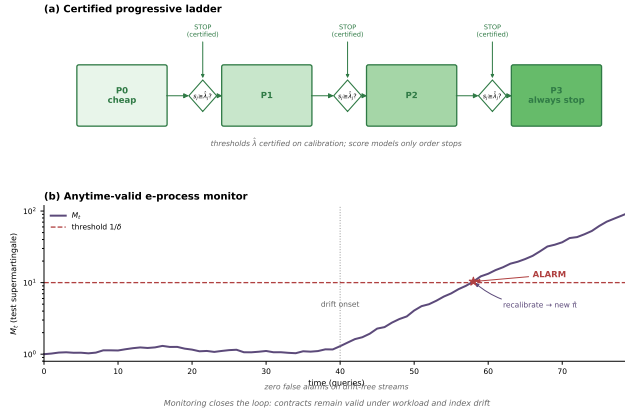


Figure 4: Certified progressive execution and the monitoring loop. Escalation thresholds carry the contract guarantee; the e-process monitor watches audited violations and, on alarm, drives safe-mode execution, changepoint-aware recalibration, and redeployment.

where the trailing term is the prior mass of restarts not yet started. The mixture weights sum to one, so M_t is itself a nonnegative supermartingale with $M_0 = 1$; an alarm fires when $M_t \geq 1/\delta$. A simple recursion maintains all started restarts of each bet jointly, so the update is $O(|W|) = O(1)$ per query in time and space, independent of t .

Proposition 3 (Anytime validity). *If the conditional risk stays within contract, $\mathbb{E}[\ell_t | \mathcal{F}_{t-1}] \leq \alpha$ for all t , then for M_t in (2), $\mathbb{P}(\exists t \leq T : M_t \geq 1/\delta) \leq \delta$.*

Seeding a fresh restart at every step is what makes the mixture practical, and it recovers the “forgetting” that a single product e-process lacks: a plain product sinks exponentially during a long compliant prefix and goes numb, whereas in (2) a change at any time τ is caught by the restart component born at τ , whose growth is unaffected by the compliant history before it. The detection delay therefore depends on the horizon but not on τ .

Proposition 4 (Detection delay). *Suppose a persistent change at τ makes $\mathbb{E}[\ell_t | \mathcal{F}_{t-1}] \geq \alpha' > \alpha$ for all $t > \tau$, and let $g^* = \max_{w \in W} \mathbb{E}[\log(1+w(\ell_t-\alpha))]$ be the best post-change log-growth rate on the grid ($g^* > 0$ whenever some $w \in W$ bets in the right direction, with $g^* = \Theta(\text{KL}(\alpha' || \alpha))$ for a grid that brackets the Kelly bet). Then for any $\delta' \in (0, 1)$, with probability at least $1 - \delta'$ the monitor alarms by time $\tau + N$ with*

$$N = O\left(\frac{\log(1/\delta) + \log(T|W|) + \log(1/\delta')}{g^*}\right),$$

a bound independent of the change time τ and of the length of the preceding compliant stretch.

Changepoint localization Alongside the mixture, CONTRACTRAG maintains one $O(1)$ CUSUM-style recursion:

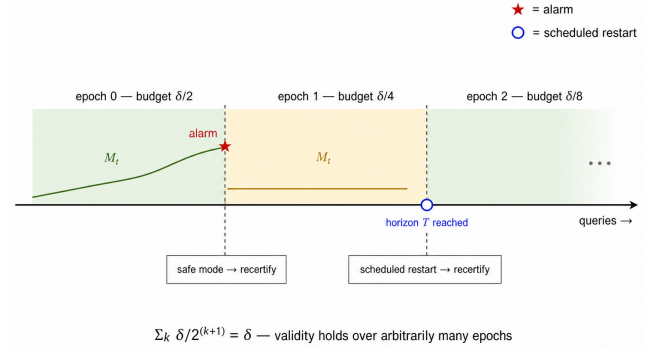


Figure 5: Lifetime operation as a sequence of finite monitoring epochs. An epoch ends either at an alarm (red star) or when its monitor exhausts the horizon T (blue circle, scheduled restart); both events trigger recertification under the next geometric budget $\delta_k = \delta/2^{k+1}$, so validity holds simultaneously over arbitrarily many epochs (Proposition 5).

the log-wealth of a single restarted product that resets whenever it falls to zero; at alarm time, the start of its currently-growing run is the standard changepoint estimate $\hat{\tau}$. Validity comes exclusively from the mixture M_t ; the recursion is a free diagnostic that tells the operator which prefix of audited losses still reflects the old regime — exactly the window recalibration must exclude.

Closed-loop certified recalibration An alarm is a statistical statement (some prefix was out of contract), not a repaired policy. On alarm, CONTRACTRAG (i) switches to *safe mode* — the strongest rung, optionally with deferral to human review. Safe mode is a best-effort fallback, not a certified policy: no plan can carry a certificate for a distribution that has not yet been sampled, and we flag it as such. (ii) It collects a recalibration window of audited post-alarm queries; (iii) re-runs the fixed-sequence walk of Section 4.5 on that window with the epoch’s certification budget; and (iv) deploys the newly certified policy — or, if the new distribution is infeasible for every candidate, reports non-certifiability and stays in flagged safe mode. The recalibration certificate is the *only* post-drift guarantee, and it is with respect to the post-change distribution, requiring only that the post-alarm window be exchangeable with what follows — drift before the alarm is irrelevant by construction. Between alarms no labels are wasted: the same audited losses double as the next calibration set.

Lifetime validity across repeated alarms A deployment does not stop after one alarm, and re-using a fixed $\delta/2$ per epoch would let error probabilities accumulate without bound; moreover, each monitor in Eq. (2) is defined for a *finite* horizon T , so unbounded operation must be built from finite blocks (Figure 5). CONTRACTRAG does both with one mechanism: the lifetime is divided into *epochs*, where an epoch ends either at an alarm or when its monitor exhausts its horizon T without alarming — a *scheduled restart* that

triggers the same recertification path as an alarm and reuses the epoch's audited losses. Epoch $k \geq 0$ receives budget $\delta_k = \delta/2^{k+1}$, split evenly between its certification ($\delta_k/2$) and its fresh horizon- T monitor ($\delta_k/2$); the *initial* certification is epoch 0's certification and therefore runs at level $\delta_0/2 = \delta/4$, not at the full δ (Algorithm 1). Since $\sum_{k \geq 0} \delta_k = \delta$, the union bound over every certification and every monitor holds over an *unbounded* horizon while every monitor operates strictly inside the finite- T regime where Propositions 3–4 apply.

Proposition 5 (Lifetime validity). *Run the alarm–recalibrate loop for arbitrarily many epochs (alarm-triggered or horizon-triggered), giving epoch k 's certification budget $\delta_k/2$ and its monitor budget $\delta_k/2$ with $\delta_k = \delta/2^{k+1}$, the initial certification being epoch 0's (level $\delta/4$). Then with probability at least $1 - \delta$, simultaneously over the whole deployment lifetime: every epoch's certified policy satisfies its contract on that epoch's distribution, and no compliant epoch raises a false alarm. By Proposition 4, the k -th monitor's detection delay bound grows only additively, by $(k + 2) \log 2 / g^*$ relative to a fixed budget δ .*

The geometric schedule prices repeated adaptation honestly: each additional epoch costs one halving of the remaining budget, which enters delays and certificate widths logarithmically rather than exhausting them (proof in A). An alternative is an infinite-horizon restart prior (weights $\propto 1/(r(r + 1))$), which removes T entirely but makes the delay bound inherit a $\log \tau$ dependence on the change time; the block construction preserves change-time independence, which is why we adopt it.

Monitoring from subsampled audits The monitor consumes audited violation indicators, and auditing costs money (an LLM judge call, an NLI entailment check, or a human label). Full-traffic auditing is not required: if each deployed query is audited independently with probability r , the audited subsequence is itself an i.i.d. draw from the deployment distribution, so every guarantee in this section applies verbatim to the audited stream — only the clock dilates, multiplying detection delays by $1/r$ in wall-clock queries. The audit rate is thus an explicit dial between monitoring cost and reaction time, and audit spend is part of the system's cost of ownership, which we meter and report in Section 6.4.

5. Experimental setup

5.1. Research questions

The evaluation is organized around six research questions:

- **RQ1 (reliability):** Which deployment methods actually keep their contract — verified directly on the *exact risk over a fixed finite population* across repeated calibration draws and across contract levels — and does the guarantee extend to identifiable query slices?

- **RQ2 (cost-efficiency):** At what cost does the certified policy meet the contract, relative to fixed pipelines, uncertified tuners, and hindsight oracles — including joint multi-dimensional contracts and strict low- α regimes with certified deferral?
- **RQ3 (risk vouchers):** Do vouchers achieve their nominal coverage over repeated calibration draws, how loose is assumption-free composition versus direct certification, and does selectivity conditioning expose real heterogeneity?
- **RQ4 (knowledge drift):** Are workload and index-staleness drifts detected, how fast, under full and sub-sampled auditing, with what lifetime guarantee across repeated alarms — and what does the auditing itself cost?
- **RQ5 (generalization):** Do the guarantees and the savings survive a change of LLM family, model developer, serving backend, and cost metric — including a locally served open-weight family metered in GPU-seconds?
- **RQ6 (planning and scalability):** Does cardinality-driven knowledge-source selection beat fixed acquisition orders, and how does certification scale to 10^5 -candidate plan spaces?

5.2. Datasets and contracts

We evaluate on four heterogeneous benchmarks with fully metered execution. (A) **HybridQA** (Chen et al., 2020): multi-hop QA over Wikipedia tables joined with entity passages; sources are a structured row-selection operator and sparse/dense passage indexes over 440K passages; quality loss $\mathbf{1}[\text{token-F1} < 0.5]$; splits train/cal/test = 5000/3000/3466 (test is the entire official development release). (B) **CRAG** (Yang et al., 2024): web QA with the official mock knowledge-graph API (five domains, dynamism labels static/slow/fast/real-time); evidence chunks carry crawl ages; answers are graded by an LLM judge with the gold answer following CRAG's protocol; loss is correctness ($\mathbf{1}[\text{not correct}]$) or hallucination ($\mathbf{1}[\text{answered and wrong}]$); splits 700/1000/1006 covering the full public development pool. (C) **ASQA** (Stelmakh et al., 2022) under the official ALCE evaluation (Gao et al., 2023a): long-form answers with citations over a pooled GTR corpus (89K passages), citation recall scored by the official TRUE-NLI entailment model (Honovich, Aharoni, Herzig, Taitelbaum, Kukliansy, Cohen, Scialom, Szpektor, Hasidim and Matias, 2022); loss $\mathbf{1}[\text{citation recall} < 50]$; splits 150/350/448 covering the full release. (D) **QAMPARI** (Amouyal et al., 2023) under the same ALCE protocol: list answers over an 86K-passage corpus; loss $\mathbf{1}[\text{citation recall} < 50]$; splits 150/350/500. QAMPARI is the adversarial regime for citation contracts — even the strongest plan violates on 61% of queries — so it probes certification exactly where point estimates are noisiest.

These scales reflect the operating regime of a deployed knowledge service that recalibrates periodically on a few

hundred to a few thousand audited queries: at $n_{\text{cal}} \in \{350, 1000, 3000\}$ and $\delta = 0.1$, the Hoeffding–Bentkus certificate half-width at mid-range α is roughly 0.075/0.04/0.025 — tight enough that certificates discriminate among ladder rungs rather than collapsing to the strongest plan. For ASQA, the smallest track, no single-split number is load-bearing: the RQ1 protocol resamples 1000 calibration/test partitions from the pooled 798 queries.

5.3. Pipelines, ladders, and cost metering

Each benchmark has a four-rung cost-ascending ladder generated by the rewrites of Section 4.2: from sparse-only retrieval with a flash-tier model up to hybrid reciprocal-rank-fusion retrieval, cross-encoder reranking, NLI verification, knowledge-graph-API evidence (CRAG), citation repair (ASQA, QAMPARI), and a max-tier model. The physical layer uses DuckDB for structured sources, BM25 for sparse retrieval, BGE embeddings with FAISS for dense retrieval, a bge-reranker-v2-m3 cross-encoder, and DeBERTa-based NLI verification; local models run on two RTX 4090 GPUs. All LLM executions are real API calls with exact token metering; reported cost is LLM spend in milli-CNY per query. A query stopping at rung j is charged rungs $0..j$ cumulatively for escalation policies and only rung j for jump policies. All LLM and API calls flow through a thread-safe cache keyed by (model, messages, decoding parameters), making every experiment reproducible and token-exact. RQ5 additionally swaps the generator column of the HybridQA ladder across four model families from four *model developers* — Alibaba’s Qwen (flash/plus/max), DeepSeek (v4-flash/v3.2/v4-pro), and Zhipu’s GLM (4.7/5/5.2), plus Google’s open-weight Gemma 3 (1b/4b/12b/27b) — served through two *serving backends*: the three commercial families are hosted on one API platform (DashScope) and token-metered at each developer’s list prices, while Gemma 3 is served locally by Ollama and metered in measured GPU-seconds priced at an RTX-4090 rental rate. We keep the two notions distinct throughout: the experiment varies the model family (and with it the price structure), not the number of API platforms. All families run over shared materialized retrieval (Section 4.3), so hosted and local plans share one cost currency.

5.4. Baselines

We compare against representative members of every deployment paradigm, all consuming identical execution matrices so comparisons are exact. *Fixed plans*: each rung as a static pipeline. *Complexity router*: Adaptive-RAG (Jeong et al., 2024) adapted to our ladders — a classifier trained to predict the cheapest correct rung. *Utility router*: per-query $\arg \max_j \hat{s}_j - \lambda \bar{c}_j$ with λ tuned to the constraint on tuning data (Ding et al., 2024; Hu et al., 2024). *BO-tuned*: Optuna (Akiba, Sano, Yanase, Ohta and Koyama, 2019) tree-structured Parzen estimation over escalation thresholds, minimizing cost subject to empirical risk $\leq \alpha$ on tuning data. *Abacus* (Russo et al., 2025): workload-level constrained selection with UCB sampling, choosing the cheapest rung whose estimated risk clears α . *Empirical-no-cert*: our own

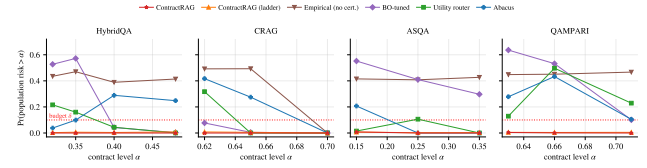


Figure 6: RQ1: probability that the deployed policy’s exact finite-population risk exceeds α , across 1000 calibration draws as the contract level varies (thirteen settings across four benchmarks). The dotted line is the certified budget $\delta = 0.1$; only CONTRACTRAG stays below it everywhere.

ladder with the cheapest thresholds whose empirical calibration risk is $\leq \alpha$ — i.e., CONTRACTRAG minus the statistical test (ablation). We report two hindsight skylines with different targets: the *zero-loss oracle* picks the per-query cheapest rung that incurs no loss (a stricter objective than any contract, hence not cost-comparable to contract-feasible methods), and the *matched oracle* is the cheapest per-query rung assignment whose *test-set* mean loss is $\leq \alpha$ — the hindsight optimum of the same constrained problem CONTRACTRAG solves, and the fair lower bound for its cost.

5.5. Evaluation protocol

For reliability (RQ1) we verify the theory’s actual claim — a bound on the probability that the deployed policy’s *true* risk exceeds α — directly, without the binomial noise of a small held-out sample. We treat the full pooled execution matrix as a fixed finite population, repeat 1000 times: draw a random calibration subset, let every method select and tune on it (scorers and routers use the fixed train split), and evaluate the selected policy’s *exact* risk on the entire population. We report $\Pr(\text{population risk} > \alpha)$ — the exact finite-population analogue of the δ -guarantee, with $\delta = 0.1$ throughout — alongside the conventional held-out estimate; at 1000 draws the two-sided 95% interval on a reported rate of 0.05 is ± 0.014 . Throughout the paper we keep three quantities distinct: (i) *certification* is an event on the calibration draw; (ii) the guarantee it purchases is on the *population* risk, with failure probability $\leq \delta$ over draws; (iii) the *test-split empirical risk* of a single draw is a noisy estimate of (ii) and may exceed α without any guarantee being violated. Tables that show a single draw are labelled accordingly. Cost-efficiency (RQ2) fixes representative contract levels per benchmark and reports realized test risk and mean cost. The remaining protocols are described with their results in Section 6.

6. Experimental results and analysis

6.1. Answer to RQ1: contract reliability

Table 1 reports the repeated-draw protocol of Section 5.5: violation here means the selected policy’s *exact* risk on the full finite population exceeds α , so Table 1 verifies the theoretical claim directly, with no test-sampling noise in the verdict; Figure 6 sweeps the contract level across

Table 1

RQ1: probability (over 1000 calibration draws) that the deployed policy's *exact risk on the full finite population* exceeds α — the quantity the certificate actually bounds — and mean cost relative to always running the strongest plan. Cells where no violating draw was observed report the one-sided 95% binomial upper bound $< .003$ rather than an absolute rate: each policy's population risk is one exact number, so draws vary only through *selection*, and $< .003$ means no draw selected an out-of-contract policy. CONTRACTRAG is the only method meeting the $\delta = 0.1$ budget on all benchmarks.

Method	HybridQA ($\alpha=.35$)		CRAG ($\alpha=.65$)		ASQA ($\alpha=.25$)		QAMPARI ($\alpha=.66$)	
	viol.	cost	viol.	cost	viol.	cost	viol.	cost
Empirical (no cert.)	.470	.19	.493	.15	.408	.45	.451	.45
BO-tuned	.572	.12	.005	.08	.410	.12	.532	.21
Utility router	.160	.07	$<.003$	.03	.106	.08	.497	.40
Abacus	.099	.34	.275	.16	$<.003$	.25	.433	.51
CONTRACTRAG (ladder)	.006	.31	.005	.38	.002	.51	.004	.75
CONTRACTRAG (full)	$<.003$	.11	$<.003$	.07	$<.003$	.43	.001	.84

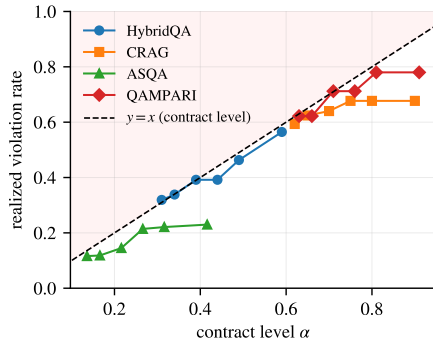


Figure 7: RQ1: target level α versus realized violation of the certified policy (test split). All points lie on or below the diagonal up to test-sampling noise: valid but tight.

thirteen settings on the same population-risk criterion. Three observations emerge. First, empirically tuned methods select policies whose *true* risk is out of contract in 41–57% of draws on their worst benchmark — these are genuine failures of the deployed policy, not artifacts of a noisy test split; across the full sweep no baseline stays below δ everywhere. Second, the certified methods hold the population-level promise with margin: the full optimizer's true risk exceeds α in at most 0.1% of draws at the headline levels — below .003 on HybridQA, CRAG, and ASQA, a single violating draw on QAMPARI, and never above 0.8% anywhere on the sweep — an order of magnitude inside the $\delta = 10\%$ budget. Rates this small are the *predicted* outcome, not an artifact: the test certifies only candidates whose calibration risk clears α by the finite-sample radius (0.02–0.07 at these n), so deploying an out-of-contract policy requires a false certification whose realized probability is far below its δ bound; where the 1000 draws contain no such event we accordingly report the binomial upper bound $< .003$, the resolution limit of the protocol, rather than an absolute rate. The same quantization explains the occasional near-floor baseline entries — e.g., the utility router at CRAG's loose $\alpha = .65$ selects only

policies whose fixed population risk happens to sit below the threshold (below the protocol's resolution here, yet 22.1% on the noisier held-out criterion, and far above δ at other levels in Figure 6) — a coincidence of one setting, not a guarantee. On the conventional held-out criterion the certified policies show 0–7% rates: the difference between the two criteria is pure test-split binomial noise, which the population protocol removes. Figure 7 shows the certificates are tight rather than vacuous: where the contract binds, realized risk tracks α with a margin of 0.02–0.07 that shrinks with calibration size, exactly the finite-sample radius of the underlying test. Third, safety is not purchased with money: on CRAG the full optimizer is both the safest and among the cheapest, because the richer certified family (jump routers) dominates threshold tuning. Abacus is safe exactly when inter-plan gaps are large relative to its sampling noise (ASQA) and unsafe when they are not (CRAG, 28%; QAMPARI, 43%) — point estimates cannot tell these regimes apart, which is the core argument for certification.

Slice-level reliability Marginal risk control can hide systematic failures on identifiable slices. On CRAG, with group-wise contract levels $\alpha_{\text{static}} = .62$, $\alpha_{\text{slow}} = .72$, $\alpha_{\text{fast}} = .92$, and $\alpha_{\text{rt}} = .93$, a single marginally certified policy violates the real-time group's contract in 99% of 200 draws (realized risk 0.99 against a 0.93 target: the cheap rungs it certifies for the average workload have no fresh evidence for real-time queries). Group-conditional certification (Section 4.4) brings every group to at most 3.5%, escalating real-time queries to the knowledge-graph-API rung while *lowering* static-group cost. When a group's contract is infeasible in the plan space — fast-changing queries at $\alpha = .65$, where even the strongest plan's group risk is 0.79 — CONTRACTRAG reports non-certifiability instead of silently under-delivering.

Answer to RQ1: *verified directly on exact finite-population risk, point-estimate feasibility is a coin flip — uncertified tuners deploy truly out-of-contract policies in up to 64% of draws — whereas certification keeps the δ -promise with an order of magnitude to spare at every contract level tested, extends it to query slices, and reports infeasibility honestly.*

6.2. Answer to RQ2: cost-efficiency at matched contracts

Table 2 and Figure 8 fix representative contract levels and show one draw each; the guarantee behind the bold rows is the population-level one verified in Table 1. At $\alpha = 0.34$ on HybridQA, CONTRACTRAG certifies a jump-router policy costing 1.05 mCNY per query with test risk 0.338 — 6.9× cheaper than the strongest plan (7.21 mCNY, risk 0.290); the uncertified methods that undercut it do so by breaking the contract (utility router: risk 0.387 $> \alpha$ at 0.40 mCNY). At $\alpha = 0.65$ on CRAG, the certified policy costs 0.30 versus 6.00 mCNY (20×); at $\alpha = 0.71$ on QAMPARI, 0.43 versus 6.49 mCNY (15×) — here certification adopts the utility router's own policy, showing that the certified family subsumes the baselines' policies and differs only in

Table 2

RQ2: single-draw test-split empirical risk / mean cost (mCNY per query) at representative contract levels. Bold: the certified policy. A certified policy's *population* risk satisfies the contract with probability $\geq 1 - \delta$ (verified in Table 1); its test-split entry here carries binomial noise of ± 0.01 – $.04$ and may sit on either side of α . The matched oracle is the cheapest hindsight per-query assignment with test mean loss $\leq \alpha$ (same objective as CONTRACTRAG); the zero-loss skyline solves a stricter objective and is not cost-comparable.

Method	HybridQA $\alpha=.34$		CRAG $\alpha=.65$		ASQA $\alpha=.216$		QAMPARI $\alpha=.71$	
	risk	cost	risk	cost	risk	cost	risk	cost
Fixed strongest	.290	7.21	.600	6.00	.116	9.01	.610	6.49
Complexity router	.509	1.46	.620	4.74	.281	1.14	.612	6.47
Utility router	.387	0.40	.625	0.28	.223	1.02	.712	0.43
BO-tuned	.380	0.97	.621	0.89	.221	1.41	.708	0.94
Abacus	.321	1.75	.600	6.00	.201	1.31	.666	1.23
Empirical (no cert.)	.372	1.45	.635	1.39	.199	5.67	.784	1.84
CONTRACTRAG (ladder)	.358	2.11	.618	2.16	.165	6.42	.724	2.88
CONTRACTRAG (full)	.338	1.05	.624	0.30	.145	6.56	.712	0.43
Matched oracle	.340	0.33	.649	0.18	.214	0.98	.710	0.30
Zero-loss skyline	.193	2.97	.491	4.74	.056	2.83	.514	5.47

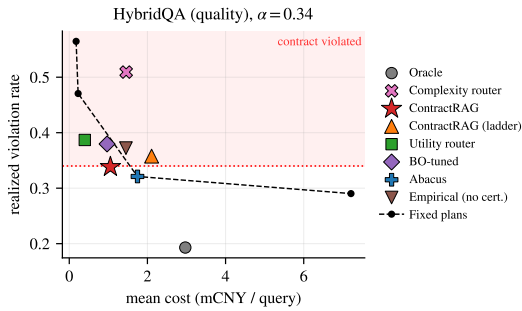


Figure 8: RQ2: cost–risk positions on HybridQA at $\alpha = 0.34$ (test split). Certified methods sit below the contract line; uncertified tuners that are cheaper sit above it.

the feasibility test. On ASQA ($\alpha = 0.216$), the certified policy's test risk is 0.145 at 6.56 versus 9.01 mCNY, while the utility router (0.223) and BO (0.221) sit above α . Several single-draw entries of certified ladder policies sit slightly above α (e.g., 0.358 vs. 0.34 on HybridQA, 0.724 vs. 0.71 on QAMPARI); these are within one standard error of test-split noise, and the repeated-draw protocol of Table 1 confirms the corresponding population risks violate in at most 0.6% of draws — what certification promises is the distributional guarantee, not that every finite sample lands below α . The fair skyline for CONTRACTRAG's objective is the *matched oracle* — the cheapest hindsight per-query assignment with test mean loss $\leq \alpha$, unattainable since it requires knowing each answer's outcome in advance. CONTRACTRAG pays 1.4–3.2 $\times$ its cost on HybridQA, CRAG, and QAMPARI, and 6.7 $\times$ on ASQA, where two-constraint feasibility forces a more conservative policy; the gap is the price of feasibility established from finite samples rather than hindsight. The zero-loss skyline row solves a different, stricter problem (no loss on any query) and is reported only for context — comparing contract-feasible costs against it is not meaningful,

and on three of four benchmarks CONTRACTRAG is in fact cheaper than it. When the contract is infeasible for every candidate, CONTRACTRAG correctly refuses to certify and falls back to best effort.

Strict contracts via certified deferral The headline levels above are dictated by what today's pipelines can achieve while answering everything; a regulated deployment wants $\alpha \leq 0.2$. With the D-family (Section 4.4), CONTRACTRAG certifies the pair $\mathbb{P}(\text{answered} \wedge \text{violating}) \leq \alpha$, $\mathbb{P}(\text{deferred}) \leq \beta$. At $\alpha = 0.1$ it certifies HybridQA at deferral budget $\beta = 0.75$ (test: answered-and-violating 0.091, deferral 0.597) and ASQA at the same budget with far less deferral used (0.087, 0.219; cost 7.64 vs. 10.90 mCNY for the strongest plan); at $\alpha = 0.2$ all four benchmarks certify — ASQA defers only 2.9% of queries at $\beta = 0.25$, CRAG needs $\beta = 0.75$ (deferral 0.529), QAMPARI 0.678 — and at $\alpha = 0.05$ no benchmark certifies at any tested $\beta \leq 0.9$, which CONTRACTRAG reports as infeasible rather than shipping a policy that cannot keep the promise. Strictness is thus purchased transparently with deferral volume, and the human-review load is itself certified, not discovered in production.

Joint multi-dimensional contracts Production contracts bind several dimensions at once. On CRAG we certify a four-constraint contract — correctness $\leq .65$, hallucination $\leq .30$, staleness $\leq .05$, latency-budget exceedance $\leq .10$ (8 s) — via the intersection–union test: the certified policy satisfies all four on test (correctness .603, hallucination .258, latency .057, all p-values $\leq 5.5 \times 10^{-3}$) at 5.02 mCNY per query; the staleness loss never fires on the 1006-query test split ($< .003$ at 95% confidence) because the certified policy escalates every stale-evidence query. IUT certifies 47 candidates against Bonferroni's 44 and selects a 3% cheaper policy — testing each constraint at full level δ instead of $\delta/4$ is uniformly more powerful. On ASQA (citation recall $\leq .25$, citation precision $\leq .35$, quality $\leq .40$, latency $\leq .15$ at 15 s) the gap is qualitative: IUT certifies 45 candidates (test risks .188/.230/.261/.033) while Bonferroni certifies *none* — at $n_{\text{cal}} = 350$ the $\delta/4$ correction is fatal, exactly the failure mode Section 4.5 predicts.

Answer to RQ2: *the guarantee costs little — the certified policy is 6.9–20 $\times$ cheaper than the strongest fixed pipeline and within 1.4–6.7 $\times$ of the matched hindsight oracle; strict contracts down to $\alpha = 0.1$ certify once deferral is priced in, four-constraint joint contracts certify with IUT where Bonferroni fails, and every method that beats CONTRACTRAG on cost does so by breaking the contract.*

6.3. Answer to RQ3: risk vouchers and composition

On HybridQA we fix a base plan (hybrid retrieval $k = 16$, rerank, plus-tier generator), apply four single rewrites, all chain prefixes, and the full composition on 500 calibration and 500 test queries (Table 3). A voucher is a confidence bound, so the correct empirical test is its coverage: pooling calibration and test into a fixed 1000-query population and

Table 3

RQ3: rewrite vouchers on HybridQA ($n_{\text{cal}} = 500$; cost in mCNY per query). Harm = loss strictly worse than the base plan's. A voucher is a $1 - \delta_r$ confidence bound, so its guarantee is *coverage over repeated calibration draws*, not dominance of any single test sample: over 1000 draws against the exact harm rate of the pooled 1000-query population, every voucher covers in $\geq 98.9\%$ of draws against a 95% nominal level (right column). The three composition rows price the *same* composed plan, so their execution columns (cost, harm) coincide by construction; the rows differ only in how the bound is built. The telescoping voucher covered in every one of the 1000 redraws, reported as the one-sided 95% lower bound $>.997$ against its nominal level of .80 ($m = 4$ steps at $\delta_r = .05$) — the deliberate price of dropping the no-synergistic-harm (NSH) assumption, which this workload violates (8% synergistic harm); the telescoping path voucher (Proposition 1) is assumption-free at 10% extra width over the (invalid) union bound.

Variant	cost	harm _{cal}	voucher $\hat{\epsilon}$	harm _{ic}	harm _{pop}	cover.
base P^*	1.74	—	—	—	—	—
R1 prefilter	1.75	.014	.026	.030	.022	.995
R2 k -trunc	1.11	.060	.081	.084	.072	.991
R3 no-rerank	1.69	.058	.078	.070	.064	.994
R4 gen-downgr.	0.33	.130	.157	.152	.141	.989
comp. (union, NSH)	0.21	.218	.342	.262	.240	—
comp. (telescoping)	0.21	.218	.376	.262	.240	$>.997$
comp. (direct cert.)	0.21	.218	.251	.262	.240	.989

redrawing the 500-query calibration set 1000 times, each single-rewrite voucher covers the population's exact harm rate in 98.9–99.5% of draws — above its 95% nominal level (Clopper–Pearson, $\delta_r = .05$), as the one-sided bound's conservatism predicts. (A single test-split column can land on either side of the voucher, as R1 and R2 do in the table; that is binomial noise, not a coverage failure.) The harm ordering — prefilter $<$ rerank-elision $\approx k$ -truncation $<$ generator downgrade — matches the intuition that evidence-preserving rewrites are safest. The composed plan cuts cost $8.3\times$ (1.74 to 0.21 mCNY) with population harm 0.240.

Composition The base-relative union bound (0.342) requires the no-synergistic-harm assumption — and the data *violate* it: 8% of harmed queries are harmed only by the composition (synergy between truncation and generator downgrade), so that bound's nominal coverage is unfounded even where its number happens to be large enough. The telescoping path voucher of Proposition 1 (step vouchers .026/.081/.105/.164, sum 0.376) costs only 10% extra width over the union bound while being assumption-free — and it covered the population chain harm in every one of the 1000 redraws ($>99.7\%$ at 95% confidence). Both remain looser than the direct certificate (0.251 mean, 98.9% coverage) on the deployed chain — which is why CONTRAC-TRAG uses vouchers to order search and certifies directly.

Selectivity conditioning The flat prefilter voucher ($\hat{\epsilon} = .026$) is marginally valid but hides heterogeneity that the planner's selectivity estimate exposes: stratifying by pool

selectivity σ , realized test harm is 0.075 on the vague-predicate stratum — three times the flat voucher — versus none observed on the fully selective stratum. The per-stratum vouchers (.076/.088/.128/.020) cover every stratum's realized harm, giving the rewrite search a risk signal that tracks *which* queries a pushdown endangers rather than an average.

Answer to RQ3: *vouchers achieve at least 98.9% repeated-draw coverage against their 95% nominal level and are correctly ordered; assumption-free telescoping composition costs only 10% width over a union bound whose assumption the data falsify; selectivity conditioning turns a marginal voucher into a per-stratum risk signal.*

6.4. Answer to RQ4: knowledge drift detection and recovery

Workload drift A stream of 1500 static/slow queries followed by 1500 fast/real-time queries (real losses, designed arrival order) breaks the marginally certified policy: violation rises from 0.58 to 0.88 against a contract of $\alpha = 0.66$, and without monitoring the system never knows (Figure 9). The e-process monitor fires at a median of 78 queries after onset (range 36–125 over 20 seeds) with no false alarms across 20 drift-free control streams — the expected observation, since the per-stream Ville bound is $\delta/2 = .05$ and a compliant stream's supermartingale in fact *decays*, keeping the realized false-alarm rate well below even that bound (confirmed at finer resolution, $< 1.5\%$, on the 200 control streams below).

Delay scaling and changepoint localization Varying the drift magnitude by mixing pools (post-change dynamic fraction $\rho \in \{.5, .75, 1\}$, 30 seeds each) makes Proposition 4 quantitatively checkable: for each magnitude we compute the post-change optimal log-growth g^* on the monitor's bet grid (.0071/.049/.137) and compare measured delays (760/203/82 queries mean) against the rate $(\log(1/\delta) + \log(T|W|))/g^*$. Mean delay sits below the rate at every magnitude, is unaffected by the 1500-query compliant prefix, and the rate is tight rather than ornamental: the p90 delay reaches 0.91–1.04 $\times$ of it at moderate and strong drifts (Figure 10). False alarms: none across the 200 drift-free control streams ($< 1.5\%$ at 95% confidence). The $O(1)$ recursion localizes the changepoint to a median error of 4 queries at $\rho = 1$ and 10 queries at $\rho = .75$ — sharp enough that recalibration can discard exactly the pre-change prefix.

Closed-loop certified recalibration Over 10 seeded streams we run the full loop: alarm $\rightarrow$ safe mode (strongest rung — a best-effort fallback that carries no certificate on the unseen post-change distribution) for a 250-query labeled window $\rightarrow$ fixed-sequence recertification on that window $\rightarrow$ deploy the newly certified policy, which is where the post-drift guarantee resides. The loop holds phase-2 violation to 0.819 versus 0.886 for the static policy, within half a point of the oracle that switches to the post-change-certified policy at the exact drift time (0.815), and beats a periodic reoptimizer

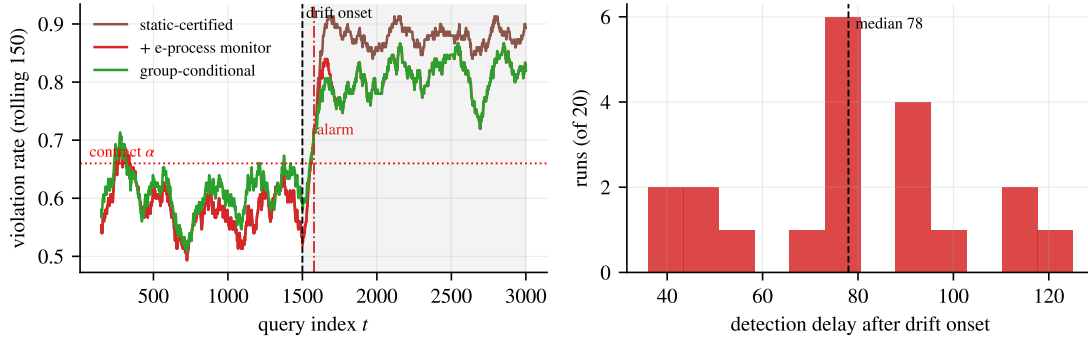


Figure 9: RQ4: workload drift on CRAG ($\alpha = 0.66$). Left: rolling violation rate; the static certified policy breaks silently after the shift at $t = 1500$; the e-process alarm (dash-dotted) triggers recalibration. Right: detection delays over 20 seeded streams; no false alarms on the 20 drift-free controls.

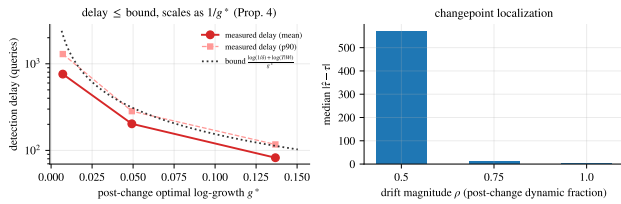


Figure 10: RQ4: measured detection delay (mean and p90 over 30 seeds per drift magnitude) versus the Proposition 4 rate $(\log(1/\delta) + \log(T|W|))/g^*$ — mean delay sits below the rate at every magnitude and the p90 tracks it within 0.6–1.0 $\times$ (left). The $O(1)$ CUSUM recursion localizes the changepoint to within a few queries for strong drifts (right). No false alarms on the 200 drift-free controls.

that rechecks every 250 queries (0.837) — while spending less than the oracle (4.77 versus 4.94 mCNY per query), because the certified post-alarm policy backs off from safe mode as soon as the window certifies a cheaper ladder.

Multi-constraint monitoring with a global error budget

Deployed contracts bind several dimensions, and an operator needs to know *which* constraint broke. We run four e-process monitors — correctness, hallucination, freshness, latency — in parallel on the certified CRAG policy, each at level $\delta_{\text{mon}}/4$ so the familywise false-alarm budget is a single global δ_{mon} . Under a fresh-to-stale index transition, the freshness monitor alarms after a median of 13.5 queries (range 5–25 over 20 seeds) while the correctness, hallucination, and latency monitors — whose constraints genuinely remain in contract — stay silent, and all four stay silent on 40 drift-free control streams. The alarm is therefore also an *attribution*: the operator learns not just that the contract broke but that the index, not the generator, is the failing knowledge source.

Lifetime operation across repeated alarms Proposition 5 prices an unbounded alarm–recalibrate loop by geometric budget spending; we exercise it on a four-phase stream (static $\rightarrow$ real-time $\rightarrow$ static $\rightarrow$ fast, 1200 queries each, $\alpha = 0.85$, 20 seeds). Runs average 1.75 alarms; the

first drift is caught in a median of 98 queries and the second — three epochs later, on a quarter of the initial budget — in 801, the logarithmic delay price the proposition predicts. Recalibration certifies a post-change policy in 84% of alarm epochs; in the remainder (the real-time phase, where even the strongest plan is out of contract) the system correctly reports non-certifiability and stays in flagged safe mode. Scheduled quiet-period rechecks recover cheap policies after drifts revert: post-drift phases run at violation 0.574 (static) against the contract while the never-recalibrating baseline ranges from 0.603 up to complete failure in the real-time phase, where every query it serves is out of contract. No false alarms occurred across 100 null streams ($< 3\%$ at 95% confidence) despite five sequential tests per run — the geometric schedule keeps the lifetime promise it proves.

The price of knowing: audit accounting The monitor consumes audited violation indicators, which are not free: on CRAG, exact-match short answers resolve 44% of audits at negligible cost, and the remaining 56% need a judge call at 0.52 mCNY — an effective 0.29 mCNY per audited query, comparable to the certified policy’s own 0.30 mCNY deployment cost. Charging it honestly: at 100% audit the certified system’s total cost of ownership (deployment + audit + amortized calibration) is 0.60 mCNY per query — the “20 $\times$ ” saving of Table 2 is 10.0 $\times$ once the meter includes the monitor. Auditing a random 25% or 10% of traffic restores the saving to 15.8 $\times$ and 17.9 $\times$ while keeping the guarantee intact on the audited subsequence (Section 4.6): measured delays dilate almost exactly as $1/r$ — medians 74 / 343 / 878 queries at $r = 1/0.25/0.1$, no false alarms at any rate — so the audit rate is a transparent dial between reaction time and monitoring spend.

Index staleness We re-execute the CRAG ladder under a 72-hour freshness contract in two index states: in sync, and one week behind. When the index falls behind *silently*, the executor keeps serving dated web evidence that is actually 168 hours older than it believes: per-rung joint (correct-and-fresh) risk jumps from (.79, .74, .71, .71) to (.94, .91, .91, .90) — even the strongest plan now violates

Table 4

RQ5: the same certified optimizer across four generator families from four model developers, served via two backends — three commercial families hosted on one API platform (DashScope, token-metered at each developer’s list prices) and open-weight Gemma-3 served locally by Ollama, metered in GPU-seconds (HybridQA, quality contract 1[F1 < 0.5], $\delta = 0.1$; splits 300/500/600 over shared materialized retrieval). “Viol. prob.” is the fraction of 200 repeated calibration/test draws whose exact finite-population risk exceeds α (must be $\leq \delta$); “emp.” is the empirical-no-certificate ablation; cells with no violating draw report the one-sided 95% binomial upper bound < .015, the protocol’s resolution at 200 draws, expected under the certificate’s built-in margin. On the noisier held-out criterion the certified rates are 4.5% (Qwen) and 9.5% (DeepSeek), still within δ . At $\alpha = 0.35$ the Gemma-3 plan space is infeasible and CONTRACTRAG reports non-certifiability instead of deploying the least-bad plan.

Family (serving)	α	metering	Strongest fixed		CONTRACTRAG (certified)				Utility router		Oracle
			risk	mCNY	risk	mCNY	saving	selected policy	risk	mCNY	
Qwen (DashScope)	.35	tokens	.300	7.29	.320	6.94	1.1×	ladder $q=.85$	.395	0.37	3.18
DeepSeek (DashScope)	.35	tokens	.295	33.51	.337	2.02	16.6×	router $\lambda=32$	.365	1.32	12.84
GLM (DashScope)	.35	tokens	.277	22.71	.317	4.06	5.6×	router $\lambda=100$	.355	3.76	13.68
Gemma-3 (local Ollama)	.45	GPU-s	.395	0.66	.432	0.57	1.2×	router $\lambda=178$	.465	0.49	1.06
Viol. prob. over 200 draws (cert. / emp.)			Qwen <.015/.300		DeepSeek <.015/.095		GLM <.015/.055		Gemma-3 <.015/<.015		(pop. risk; $\delta=0.1$)

$\alpha = 0.75$, so no amount of static escalation helps. The static certified policy runs at phase-2 violation 0.926 and never knows; the monitor alarms after a median of 117 queries (no false alarms on the 20 in-sync controls) and switches to the policy certified on stale-state calibration data — a freshness-honest re-execution whose filter knows the lag, drops out-of-contract evidence, and escalates to the still-fresh knowledge-graph API — restoring violation to $0.759 \approx \alpha$ at 2.52 mCNY per query, within 1% of the oracle that recalibrates exactly at the drift point. Freshness contracts are thus operationally different from quality contracts: staleness breaks every plan at once, and only detect-and-recertify — not conservative planning — restores the guarantee.

Answer to RQ4: *drift is detected in tens of queries with no false alarms and attributed to the failing constraint, the measured delays match the proven bound at full and subsampled audit rates, certified recalibration restores the contract within 1% of an oracle that knows the drift time, the geometric budget schedule sustains repeated alarms over a full lifetime, and the auditing that powers all of this costs at most half of the claimed savings — 10–18× instead of 20×, reported honestly.*

6.5. Answer to RQ5: generalization across LLM families and serving backends

A fair objection to any cost-optimization result is that it may be an artifact of one vendor’s price sheet. We therefore hold the entire framework fixed — retrieval ladder, sufficiency scorers, candidate families, certification — and swap only the generator column across four model families: Qwen (Qwen Team, 2024), DeepSeek (DeepSeek-AI, 2024, 2025), GLM (GLM Team, 2024), and open-weight Gemma-3 (Gemma Team, 2025) (Table 4, Figure 11). What this design varies is the model developer and its price structure (four developers; two serving backends: one hosted API platform and one local deployment); testing robustness across independent hosting *platforms* for the same model would require replicating the ladder on a second API vendor, which we leave outside scope and do not claim. Four results stand out. (i) *The guarantee transfers*: on every family where the

contract is feasible, over 200 repeated calibration/test draws per family the certified policy’s exact population risk never exceeds α in any draw (below 1.5% at 95% confidence on all four families against the $\delta = 0.1$ budget; up to 9.5% on the noisier held-out criterion), while the empirical-no-certificate ablation deploys truly out-of-contract policies in up to 30% of draws and the tuned utility router’s single deployment exceeds α on all four families. Uncertified selection does not fail on one vendor’s quirk; it fails everywhere. (ii) *The savings grow with the family’s price spread*: certified cost drops 16.6× for DeepSeek and 5.6× for GLM against each family’s strongest plan, and the certified policy family itself changes with the price structure — a threshold ladder for Qwen’s smooth tiering, jump routers for DeepSeek and GLM, whose flash tiers are strong but whose top tiers are disproportionately expensive. The optimizer discovers this from data; nothing in the system encodes a vendor. (iii) *Infeasibility is reported, not hidden*: at $\alpha = 0.35$ no Gemma-3 plan can meet the contract, and CONTRACTRAG returns non-certifiable instead of deploying the least-bad plan. (iv) *Cost-metric generality*: Gemma plans are metered in measured GPU-seconds, and certification is indifferent to which currency the cost column uses — hosted token pricing and local GPU pricing flow through the identical machinery.

Answer to RQ5: *the guarantee and the savings are properties of the framework, not of a vendor: the population-risk guarantee holds without a single violating draw on all four families, savings reach 16.6× where price spreads are wide, and the certified policy structure adapts to each family’s economics.*

6.6. Answer to RQ6: knowledge-source selection and scalability

Acquisition-order selection On HybridQA we execute four physical plans for the table–text join at a fixed flash-tier generator, so plan differences are purely retrieval-side: {table-first, text-first} $\times$ {lexical, hybrid+rerank}, with GPU-priced retrieval included in cost. Estimated pool selectivity σ , computed from row-link cardinalities *before* execution, stratifies violation cleanly and monotonically for every

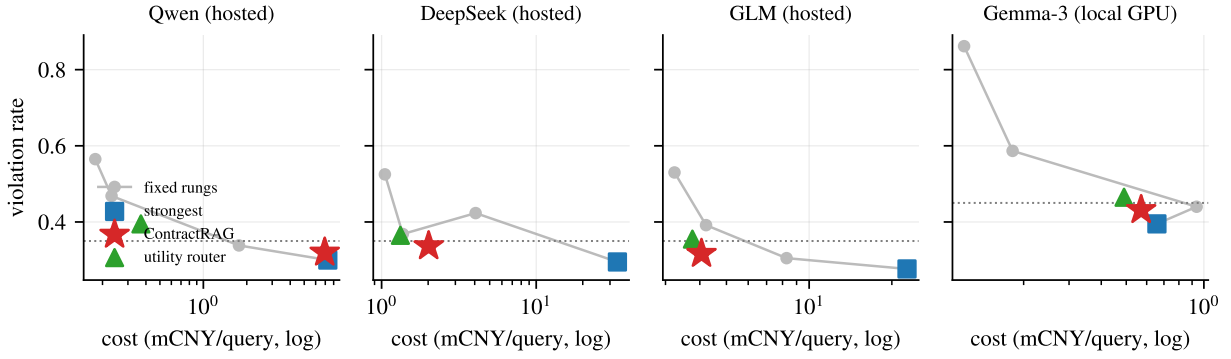


Figure 11: RQ5: the same certified optimizer across four generator families (HybridQA, $\alpha = 0.35$; Gemma at $\alpha = 0.45$). Retrieval is materialized once and shared; only the generator column changes. Grey: the four fixed rungs; star: the certified policy. The optimizer selects a different policy family per price structure and stays below the contract line everywhere it certifies.

path: text-first-hybrid moves from 0.511 (lowest- σ tercile) to 0.364 (highest), and the lexical paths pay a violation premium over hybrid that grows with σ — the selectivity-dependent crossover that source selection exploits. Feeding the chooser family (fixed paths plus selectivity-threshold rules) to the certification layer at $\alpha = 0.5$ certifies a mixed policy at 0.337 mCNY per query — 31% below the strongest fixed path (0.492) with realized test violation $0.481 \leq \alpha$. Everything a relational optimizer does with selectivities — pick the join order, skip the expensive operator when the predicate is strong — transfers to knowledge acquisition, except that feasibility is established by the test, not by the estimate.

Plan-space scalability and overhead Certification is a vectorized pass over the materialized execution matrix, so enlarging the plan space is cheap: growing the threshold grid from 69 to 97,365 candidates raises certification wall-clock to 8.5 s on HybridQA, 4.1 s on CRAG, and 0.9 s on ASQA — still below the cost of a single max-tier LLM call, with selection-proofness (Theorem 1) guaranteeing that the extra $10^3\times$ candidates cost nothing in validity. The default 69-candidate space certifies in 5–8 ms; overall optimization overhead is below 10^{-3} of a single LLM call, and the online policy adds one score lookup per rung plus an $O(1)$ monitor update per query. Materializing retrieval once and sharing it across the four generator families of RQ5 removes three quarters of retrieval compute and eliminates GPU contention between the reranker and locally served generators. One caveat: more candidates are statistically free for *safety* but not automatically better for *power* — at $n_{\text{cal}} = 500$, a dense 8,029-candidate grid reaches the contract boundary sooner on Qwen and certifies nothing, which is why CONTRACTRAG ships the compact curated space as the default.

Answer to RQ6: *selectivity-driven source selection cuts retrieval-side cost 31% under the same certificate, and certification scales to 10^5 -candidate plan spaces in seconds with provably unchanged validity.*

7. Discussion

7.1. Why does certification win, and when is it free?

The experiments locate the failure of point-estimate tuning precisely: selecting the cheapest configuration whose measured risk clears a threshold is an uncorrected multiple-testing procedure, so its winner is systematically the configuration whose noise broke in its favor — the winner’s curse. The effect is strongest exactly where adaptive systems are most attractive: many candidates, small quality gaps between adjacent tiers, and moderate audit budgets. Certification neutralizes the curse not by testing less but by testing in a data-independent order and stopping at the first failure, which caps the family-wise error at δ regardless of how many candidates follow. The cost of the guarantee is a finite-sample margin of 0.02–0.07 at practical calibration sizes — and on benchmarks where the certified family is richer than the baselines’ (CRAG), the certified policy is simultaneously the safest and among the cheapest, i.e., the guarantee is free. The margin shrinks at the standard $O(n^{-1/2})$ rate, so operators can trade audit budget for cost directly.

7.2. Implications for knowledge-based systems

Three design lessons generalize beyond RAG. First, *separate the promise from the search*: learned components (sufficiency scores, vouchers, selectivity models) should influence only which candidate is examined next, never whether a candidate is deployable; this division lets a system exploit arbitrarily aggressive heuristics without inheriting their failure modes. Second, *knowledge acquisition is a plan-selection problem*: the selectivity abstraction that drives relational access-path selection transfers to choosing between structured and unstructured knowledge sources, and the same estimates that pick the plan can index risk bounds. Third, *a certificate without a monitor is a snapshot*: freshness drift breaks every plan at once, so no amount of conservative static planning substitutes for anytime-valid detection plus recertification. We expect these principles to apply to knowledge-graph question answering, tool-using

agents, and other knowledge services whose components are stochastic and priced.

7.3. Limitations and future directions

We acknowledge several limitations. (i) The guarantee is relative to the audited loss: a systematically biased auditor (an LLM judge that overrates fluent answers) shifts what is being certified; the machinery accepts human audits without modification, and the audit accounting of Section 6.4 shows their cost is a first-order term that must be (and in our tables is) charged against the savings. (ii) Certification assumes the calibration sample is exchangeable with deployment within an epoch; adversarial or rapidly cycling drift would trigger frequent recalibration, and although the geometric spending schedule keeps lifetime validity, each cycle halves the remaining budget and lengthens detection delays logarithmically — safe mode between an alarm and the next successful recertification is best-effort, not certified. (iii) The fixed-sequence procedure returns the cheapest policy of its certified prefix; this coincides with the family-wide cheapest feasible policy only under the ordering conditions of Proposition 2, and the dense-grid experiment shows the power cost of violating them. (iv) The policy families we certify are built over a linear ladder plus routers; richer spaces (per-operator adaptive plans, multi-agent pipelines) enlarge the search but also stress the power of a fixed-sequence walk at small audit budgets. (v) Group-conditional certification requires groups known at query time and pays a Bonferroni factor; overlapping or learned groups are future work. (vi) Our cost model is monetary; extending contracts to energy or carbon budgets is mechanical but unexplored.

7.4. Threats to validity

Internal validity: all methods consume identical execution matrices, so comparisons cannot be confounded by retrieval variance; the repeated-draw protocol (1000 draws) evaluates each selected policy on the *exact* finite population, so the reported violation probabilities carry Monte Carlo error of about ± 0.014 and no test-sampling error at all. *Construct validity:* losses follow each benchmark’s official protocol (token-F1, CRAG’s LLM-judge grading, ALCE’s TRUE-NLI citation entailment), whose agreement with human raters is documented by their authors; the certificate is exact for these losses, and Section 7 discusses auditor bias. *External validity:* results cover four benchmarks, four LLM families from four model developers, two serving backends (one hosted API platform, one local deployment), and two cost currencies, but all workloads are English-language QA; agentic and multilingual workloads may distribute difficulty differently. *Reproducibility:* every LLM call is cached token-exactly, and all numbers regenerate from the released artifact without further API calls.

8. Conclusion

This paper reframed the deployment of retrieval-augmented generation as a certified knowledge-service problem. Evidence contracts give applications a declarative vocabulary for reliability — admissible violation rates for answer quality, citation completeness, evidence freshness, and latency — and CONTRACTRAG makes the vocabulary enforceable: plan rewrites are priced by assumption-free risk vouchers, progressive knowledge-acquisition policies are searched over a plan lattice, feasibility is established by a distribution-free fixed-sequence test whose guarantee survives unlimited search, and an anytime-valid e-process monitor defends the certificate against knowledge drift with certified recalibration, geometric error spending across repeated alarms, and explicitly metered audit costs. Across four heterogeneous benchmarks and four LLM families from four model developers, certification was the only mechanism that kept the declared promise — verified on exact finite-population risk, at most 0.8% violation probability against a 10% budget, where uncertified tuning violated in up to 64% of draws — while remaining up to 20× cheaper than the strongest fixed pipeline before audit costs and 10–18× after. We believe contract-driven certification offers a principled and practical path for knowledge-based systems that must be simultaneously economical and accountable, and we release all execution matrices and code to support replication and extension.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All code, execution matrices, calibration artifacts, and the token-exact LLM-call cache are publicly available at <https://github.com/xkk9866/contractRagDB>; every number in the paper regenerates from the artifact without further LLM calls.

CRedit authorship contribution statement

Kang Yao: Conceptualization, Methodology, Software, Formal analysis, Writing – original draft. **Congcong Tian:** Software, Validation, Investigation, Data curation. **Quanbo Cheng:** Investigation, Visualization, Writing – review & editing. **Shengjiang Zhang:** Validation, Data curation, Writing – review & editing. **Hao Yin:** Investigation, Resources, Writing – review & editing. **Jinjiang Cui:** Conceptualization, Supervision, Project administration, Writing – review & editing. **Weiwei Fu:** Supervision, Funding acquisition, Writing – review & editing.

References

- Aarab, I., 2025. LLM-based IR-system for bank supervisors. *Knowledge-Based Systems* 310, 112914.

- Aggarwal, P., Madaan, A., Anand, A., Potharaju, S.P., Mishra, S., Zhou, P., Gupta, A., Rajagopal, D., Kappaganthu, K., Yang, Y., Upadhyay, S., Faruqui, M., Mausam, 2024. Automix: Automatically mixing language models, in: Advances in Neural Information Processing Systems (NeurIPS).
- Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M., 2019. Optuna: A next-generation hyperparameter optimization framework. KDD.
- Amouyal, S.J., Wolfson, T., Rubin, O., Yoran, O., Herzig, J., Berant, J., 2023. QAMPARI: A benchmark for open-domain questions with many answers from multiple paragraphs, in: Proceedings of the Third Workshop on Natural Language Generation, Evaluation, and Metrics (GEM).
- Angelopoulos, A.N., Bates, S., Candès, E.J., Jordan, M.I., Lei, L., 2025. Learn then test: Calibrating predictive algorithms to achieve risk control. Annals of Applied Statistics.
- Angelopoulos, A.N., Bates, S., Fisch, A., Lei, L., Schuster, T., 2024. Conformal risk control. ICLR.
- Asai, A., Wu, Z., Wang, Y., Sil, A., Hajishirzi, H., 2024. Self-RAG: Learning to retrieve, generate, and critique through self-reflection, in: ICLR.
- Bentkus, V., 2004. On Hoeffding's inequalities. Annals of Probability 32.
- Berger, R.L., 1982. Multiparameter hypothesis testing and acceptance sampling. Technometrics 24.
- Budur, E., Özgelik, R., Soylu, D., Khattab, O., Güngör, T., Potts, C., 2024. Building efficient and effective OpenQA systems for low-resource languages. Knowledge-Based Systems 302, 112243.
- Chen, J., Lin, H., Han, X., Sun, L., 2024. Benchmarking large language models in retrieval-augmented generation, in: Proceedings of the AAAI Conference on Artificial Intelligence.
- Chen, L., Zaharia, M., Zou, J., 2023. FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv:2305.05176.
- Chen, W., Zha, H., Chen, Z., Xiong, W., Wang, H., Wang, W.Y., 2020. HybridQA: A dataset of multi-hop question answering over tabular and textual data, in: Findings of EMNLP.
- Cherian, J.J., Gibbs, I., Candès, E.J., 2024. Large language model validity via enhanced conformal prediction methods, in: Advances in Neural Information Processing Systems (NeurIPS).
- DeepSeek-AI, 2024. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437.
- DeepSeek-AI, 2025. Deepseek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv preprint arXiv:2501.12948.
- Deng, J., Shi, K., Wu, Z., Huo, H., Wang, D., Xu, G., 2026. Enabling collaborative parametric knowledge calibration for retrieval-augmented vision question answering. Knowledge-Based Systems 346.
- Ding, D., Mallick, A., Wang, C., Sim, R., Mukherjee, S., Rühle, V., Lakshmanan, L.V.S., Awadallah, A.H., 2024. Hybrid LLM: Cost-efficient and quality-aware query routing, in: International Conference on Learning Representations (ICLR).
- Es, S., James, J., Espinosa-Anke, L., Schockaert, S., 2024. RAGAs: Automated evaluation of retrieval augmented generation, in: EACL.
- Farquhar, S., Kossen, J., Kuhn, L., Gal, Y., 2024. Detecting hallucinations in large language models using semantic entropy. Nature 630, 625–630.
- Fisch, A., Schuster, T., Jaakkola, T., Barzilay, R., 2021. Efficient conformal prediction via cascaded inference with expanded admission. ICLR.
- Gao, T., Yen, H., Yu, J., Chen, D., 2023a. Enabling large language models to generate text with citations, in: EMNLP.
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., Wang, H., 2023b. Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997.
- Gemma Team, 2025. Gemma 3 technical report. arXiv preprint arXiv:2503.19786.
- GLM Team, 2024. ChatGLM: A family of large language models from GLM-130B to GLM-4 all tools. arXiv preprint arXiv:2406.12793.
- Grünwald, P., de Heide, R., Koolen, W.M., 2024. Safe testing. Journal of the Royal Statistical Society Series B: Statistical Methodology 86, 1091–1128.
- Gui, Y., Jin, Y., Ren, Z., 2024. Conformal alignment: Knowing when to trust foundation models with guarantees, in: Advances in Neural Information Processing Systems (NeurIPS).
- Honovich, O., Aharoni, R., Herzig, J., Taitelbaum, H., Kukliansy, D., Cohen, V., Scialom, T., Szpektor, I., Hassidim, A., Matias, Y., 2022. TRUE: Re-evaluating factual consistency evaluation, in: NAACL.
- Hu, Q.J., Bieker, J., Li, X., Jiang, N., Keigwin, B., Ranganath, G., Keutzer, K., Upadhyay, S.K., 2024. Routerbench: A benchmark for multi-LLM routing system. arXiv preprint arXiv:2403.12031.
- Jeong, S., Baek, J., Cho, S., Hwang, S.J., Park, J.C., 2024. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity, in: NAACL.
- Jiang, Z., Xu, F.F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., Neubig, G., 2023. Active retrieval augmented generation, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP).
- Kabra, N., DeWitt, D.J., 1998. Efficient mid-query re-optimization of sub-optimal query execution plans, in: SIGMOD.
- Kang, M., Gürel, N.M., Yu, N., Song, D., Li, B., 2024. C-RAG: Certified generation risks for retrieval-augmented language models, in: ICML.
- Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T.T., Moazam, H., Miller, H., Zaharia, M., Potts, C., 2024. DSPy: Compiling declarative language model calls into self-improving pipelines, in: ICLR.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., tau Yih, W., Rocktäschel, T., Riedel, S., Kiela, D., 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks, in: NeurIPS.
- Li, S., Park, S., Lee, I., Bastani, O., 2024. TRAQ: Trustworthy retrieval augmented question answering via conformal prediction, in: NAACL.
- Liu, C., Russo, M., Cafarella, M., Cao, L., Chen, P.B., Chen, Z., Franklin, M., Kraska, T., Madden, S., Vitagliano, G., 2025. A declarative system for optimizing AI workloads, in: CIDR.
- Markl, V., Raman, V., Simmen, D., Lohman, G., Pirahesh, H., Cilimdžic, M., 2004. Robust query processing through progressive optimization, in: SIGMOD.
- Mohri, C., Hashimoto, T., 2024. Language models with conformal factuality guarantees, in: International Conference on Machine Learning (ICML).
- Ong, I., Almahairi, A., Wu, V., Chiang, W.L., Wu, T., Gonzalez, J.E., Kadous, M.W., Stoica, I., 2024. RouteLLM: Learning to route LLMs with preference data. arXiv:2406.18665.
- Patel, L., Jha, S., Guestrin, C., Zaharia, M., 2024. LOTUS: Enabling semantic queries with LLMs over tables of unstructured and structured data. arXiv:2407.11418.
- Quach, V., Fisch, A., Schuster, T., Yala, A., Sohn, J.H., Jaakkola, T.S., Barzilay, R., 2024. Conformal language modeling, in: International Conference on Learning Representations (ICLR).
- Qwen Team, 2024. Qwen2.5 technical report. arXiv preprint arXiv:2412.15115.
- Ramdas, A., Grünwald, P., Vovk, V., Shafer, G., 2023. Game-theoretic statistics and safe anytime-valid inference. Statistical Science 38.
- Ru, D., Qiu, L., Hu, X., Zhang, T., Shi, P., Chang, S., Jiayang, C., Wang, C., Sun, S., Li, H., Zhang, Z., Wang, B., Jiang, J., He, T., Wang, Z., Liu, P., Zhang, Y., Zhang, Z., 2024. RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation, in: Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track.
- Russo, M., Sudhir, S., Vitagliano, G., Liu, C., Kraska, T., Madden, S., Cafarella, M., 2025. Abacus: A cost-based optimizer for semantic operator systems. PVLDB 18.
- Saad-Falcon, J., Khattab, O., Potts, C., Zaharia, M., 2024. ARES: An automated evaluation framework for retrieval-augmented generation systems, in: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL).
- Schuster, T., Fisch, A., Gupta, J., Dehghani, M., Bahri, D., Tran, V.Q., Tay, Y., Metzler, D., 2022. Confident adaptive language modeling. NeurIPS.

- Shafer, G., 2021. Testing by betting: A strategy for statistical and scientific communication. JRSS-A 184.
- Shankar, S., Parameswaran, A.G., Wu, E., 2024. DocETL: Agentic query rewriting and evaluation for complex document processing. arXiv:2410.12189.
- Shin, J., Ramdas, A., Rinaldo, A., 2024. E-detectors: A nonparametric framework for sequential change detection. The New England Journal of Statistics in Data Science 2.
- Stelmakh, I., Luan, Y., Dhingra, B., Chang, M.W., 2022. ASQA: Factoid questions meet long-form answers, in: EMNLP.
- Ville, J., 1939. Étude critique de la notion de collectif. Gauthier-Villars.
- Xu, F., Shi, W., Choi, E., 2024. RECOMP: Improving retrieval-augmented LMs with compression and selective augmentation, in: International Conference on Learning Representations (ICLR).
- Yan, S.Q., Gu, J.C., Zhu, Y., Ling, Z.H., 2024. Corrective retrieval augmented generation. arXiv preprint arXiv:2401.15884.
- Yang, W., Some, L., Bain, M., Kang, B., 2025. A comprehensive survey on integrating large language models with knowledge-based methods. Knowledge-Based Systems 318, 113503.
- Yang, X., Sun, K., Xin, H., Sun, Y., Bhalla, N., Chen, X., Choudhary, S., Gui, R.D., Jiang, Z.W., Jiang, Z., et al., 2024. CRAG – comprehensive RAG benchmark, in: NeurIPS Datasets and Benchmarks.
- Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., Zhang, H., Gonzalez, J.E., Stoica, I., 2023. Judging LLM-as-a-judge with MT-Bench and chatbot arena, in: Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track.
- Zhou, Z., Li, M., Fang, X., Zhou, X., Lin, W., Zheng, Z., 2026. LGMT: Logic-grounded metamorphic testing for evaluating the reasoning reliability of LLMs. Knowledge-Based Systems 348, 116324.

A. Proofs

A.1. Proof of Theorem 1

Fix the candidate order $\pi_{(1)}, \pi_{(2)}, \dots$ (a function of the train split only, hence independent of the calibration sample). For candidate π let $H_0(\pi) = \bigcup_j \{R_j(\pi) > \alpha_j\}$ be its composite null. The per-candidate test rejects iff every constraint's Hoeffding–Bentkus p-value is $\leq \delta$. Each $p_\pi^{(j)}$ is a valid p-value for $H_0^{(j)}$ (Bentkus, 2004; Angelopoulos et al., 2025); by the intersection–union principle (Berger, 1982), when $H_0(\pi)$ holds at least one $H_0^{(j)}$ holds, so $\mathbb{P}(\text{reject } \pi) \leq \mathbb{P}(p_\pi^{(j^*)} \leq \delta) \leq \delta$ for a true null j^* : the composite test has size $\leq \delta$. Let k^* be the index of the first infeasible candidate in the order (if none, no error is possible). The procedure deploys an infeasible candidate only if the walk certifies $\pi_{(k^*)}$ (it cannot reach later candidates without doing so, and all earlier candidates are feasible). That event has probability $\leq \delta$. Choosing the cheapest *certified* candidate only ever selects among candidates that were certified, so the error event is unchanged. $\square$

A.2. Proof of Proposition 1

Write $\rho_i^{\text{step}} = \mathbb{P}_q(\ell(P_i, q) > \ell(P_{i-1}, q))$ for the stepwise harm of R_i against its predecessor. *Deterministic inclusion*: for any query q with $\ell(P_m, q) > \ell(P_0, q)$, the telescoping sum $\sum_{i=1}^m [\ell(P_i, q) - \ell(P_{i-1}, q)] = \ell(P_m, q) - \ell(P_0, q) > 0$ forces at least one strictly positive summand, i.e., $q \in \{\ell(P_i) > \ell(P_{i-1})\}$ for some i . Hence $\{\ell(P_m) > \ell(P_0)\} \subseteq \bigcup_i \{\ell(P_i) > \ell(P_{i-1})\}$ pointwise, with no assumption on the rewrites. Taking probabilities, $\rho(R_m \cdots R_1; P_0) \leq \sum_i \rho_i^{\text{step}}$.

Each step voucher satisfies $\mathbb{P}(\rho_i^{\text{step}} > \hat{\epsilon}_i^{\text{step}}) \leq \delta_r$ (Clopper–Pearson); on the event that all m bounds hold (probability $\geq 1 - m\delta_r$ by a union bound over the m vouchers, the only stochastic step), $\rho(R_m \cdots R_1; P_0) \leq \sum_i \hat{\epsilon}_i^{\text{step}}$. $\square$

Remark 2. The base-relative union bound (each rewrite vouchered against P_0) requires the additional no-synergistic-harm assumption $\{\ell(P_m) > \ell(P_0)\} \subseteq \bigcup_i \{\ell(R_i(P_0)) > \ell(P_0)\}$, which composed rewrites violate on real workloads (Section 6.3); the telescoping decomposition replaces that behavioral assumption with an algebraic identity by conditioning each voucher on the actual chain prefix.

A.3. Proof of Proposition 3

Under the null $\mathbb{E}[\ell_t | \mathcal{F}_{t-1}] \leq \alpha$, each per-step factor satisfies $\mathbb{E}[1 + w(\ell_t - \alpha) | \mathcal{F}_{t-1}] = 1 + w(\mathbb{E}[\ell_t | \mathcal{F}_{t-1}] - \alpha) \leq 1$ for any $w \in (0, 1/\alpha)$ and $\ell_t \in [0, 1]$ (the factor is also nonnegative since $w < 1/\alpha$ and $\ell_t \geq 0$). Hence each restarted product $E_t^{(r,w)} = \prod_{i=r}^t (1 + w(\ell_i - \alpha))$ is a nonnegative supermartingale, equal to 1 for $t < r$, so $\mathbb{E}[E_t^{(r,w)} | \mathcal{F}_{t-1}] \leq E_{t-1}^{(r,w)}$ for all t . The statistic in (2) is a fixed convex combination of these supermartingales and the constants 1 (the not-yet-started restarts $r > t$, each contributing $\frac{1}{T|W|} \cdot 1$, which total $1 - t/T$). A convex combination of nonnegative supermartingales, with weights fixed in advance and summing to one, is again a nonnegative supermartingale, and $M_0 = 1$. This is exactly where the mixture is needed: a *maximum* $\max_r E_t^{(r,w)}$ is not a supermartingale (the maximizing index is chosen with hindsight), so Ville’s inequality would not apply to it. Applying Ville’s inequality (Ville, 1939) to M_t gives $\mathbb{P}(\exists t \leq T : M_t \geq 1/\delta) = \mathbb{P}(\sup_{t \leq T} M_t \geq 1/\delta) \leq \mathbb{E}[M_0] \delta = \delta$. $\square$

A.4. Proof of Proposition 4

Consider the single mixture component with restart $r = \tau + 1$ and the growth-optimal grid bet w^* , the maximizer of $\mathbb{E}[\log(1 + w(\ell_t - \alpha))]$ over $w \in W$, whose post-change per-step log-growth is $g^* > 0$. Its increments $X_i = \log(1 + w^*(\ell_i - \alpha))$ are bounded ($|X_i| \leq \log \frac{1}{1-w^*g} \vee \log(1 + w^*(1 - \alpha)) =: b$) with conditional mean $\geq g^*$ for $i > \tau$. By the Azuma–Hoeffding bound applied to $\sum_{i=\tau+1}^{\tau+n} (X_i - g^*)$, with probability $\geq 1 - \delta'$,

$$\log E_{\tau+n}^{(\tau+1, w^*)} = \sum_{i=\tau+1}^{\tau+n} X_i \geq ng^* - b\sqrt{2n \log(1/\delta')}.$$

Since $M_{\tau+n} \geq \frac{1}{T|W|} E_{\tau+n}^{(\tau+1, w^*)}$, the alarm threshold $M_{\tau+n} \geq 1/\delta$ is reached once $ng^* - b\sqrt{2n \log(1/\delta')} \geq \log(1/\delta) + \log(T|W|)$. Solving this scalar inequality gives

$$n = O((\log(1/\delta) + \log(T|W|) + \log(1/\delta'))/g^*),$$

independent of τ . For a Bernoulli loss the Kelly bet $w^* = \frac{\alpha' - \alpha}{\alpha'(1 - \alpha)}$ attains $g^* = \text{KL}(\alpha' \| \alpha)$; any grid W bracketing it loses only a constant factor, giving the stated $\text{KL}(\alpha' \| \alpha)$ rate. $\square$

A.5. Proof of Proposition 2

Let $\pi^\dagger$ be the cheapest candidate with $R(\pi^\dagger) \leq \alpha - \gamma$, at position K in the walk, with all true risks non-decreasing along the order up to K . *Certification of the prefix.* For every $\pi_{(k)}$, $k \leq K$: $R(\pi_{(k)}) \leq \alpha - \gamma$, and by Hoeffding the empirical risk is $\leq \alpha - \gamma/2$ with probability $1 - e^{-n\gamma^2/2}$; the Hoeffding–Bentkus p-value at empirical risk $\alpha - \gamma/2$ is $\leq e^{-n h_1(\alpha - \gamma/2, \alpha)} \leq e^{-n\gamma^2/2}$, which is $\leq \delta$ once $n \geq 2\gamma^{-2} \log(1/\delta)$. A union bound over the K candidates (at level δ'/K each) shows all are certified simultaneously with probability $\geq 1 - \delta'$ when $n \geq 2\gamma^{-2}(\log(1/\delta) \vee \log(K/\delta'))$; on this event the walk reaches at least position K and $\pi^\dagger$ is in the certified set. *Uniform cost estimation.* The walk does not stop at K — it continues until the first failed test — so the deployed policy may sit at any position in the certified set, whose extent is data-dependent and not known in advance. We therefore control cost estimates uniformly over all M candidates: per-query costs lie in $[0, C_{\max}]$, so by Hoeffding plus a union bound over the M candidates (at level δ'/M each), with probability $\geq 1 - \delta'$ every candidate's estimated mean calibration cost $\hat{c}(\pi)$ satisfies $|\hat{c}(\pi) - c(\pi)| \leq \epsilon_n := C_{\max} \sqrt{\log(2M/\delta')/(2n)}$ simultaneously — in particular for $\hat{\pi}$ and $\pi^\dagger$, wherever the walk terminates. The deployed $\hat{\pi}$ minimizes $\hat{c}$ over the certified set, which contains $\pi^\dagger$, hence $c(\hat{\pi}) \leq \hat{c}(\hat{\pi}) + \epsilon_n \leq \hat{c}(\pi^\dagger) + \epsilon_n \leq c(\pi^\dagger) + 2\epsilon_n$. Both events hold together with probability $\geq 1 - 2\delta'$. (If the cheapest-selection is restricted to the first K walk positions, or costs are estimated on an independent split, the union bound over M shrinks to K and ϵ_n improves accordingly; validity is unaffected either way.) Without the monotone-order premise the walk may stop before K ; the certified-prefix optimality and the $2\epsilon_n$ cost bound continue to hold with $\pi^\dagger$ replaced by the cheapest feasible candidate of the certified prefix, and validity is unaffected. $\square$

A.6. Proof of Proposition 5

Index epochs $k = 0, 1, 2, \dots$ in order of their terminating events — an alarm or the exhaustion of the monitor's horizon T (scheduled restart). Each epoch lasts at most T queries, so an unbounded lifetime is covered by countably many epochs, and each epoch's monitor is a fresh finite-horizon detector run strictly within its own block, the regime in which Proposition 3 holds. Epoch k 's certification is a fixed-sequence test at level $\delta_k/2$ run on data from that epoch alone — the initial certification is epoch 0's, run at $\delta_0/2 = \delta/4$ on the initial calibration split, so it is inside the schedule rather than an extra δ -sized event on top of it — and by Theorem 1 the probability that epoch k deploys a policy violating its contract (on the epoch's distribution) is at most $\delta_k/2$; epoch k 's monitor is an e-detector at level $\delta_k/2$, so by Proposition 3 the probability that it false-alarms while the epoch is compliant is at most $\delta_k/2$. Horizon-triggered boundaries consume no additional error probability: exhausting T without alarm is a deterministic event, and the subsequent recertification is already accounted at level $\delta_{k+1}/2$. The lifetime error event is the union over k of these 2 events per epoch; by countable subadditivity its probability

is at most $\sum_{k \geq 0} \delta_k = \sum_{k \geq 0} \delta/2^{k+1} = \delta$, with no bound on the number of epochs. For the delay claim, the k -th monitor's alarm threshold is $\log(2^{k+2}/\delta) = \log(1/\delta) + (k+2)\log 2$, which enters the bound of Proposition 4 additively, giving the stated $(k+2)\log 2/g^*$ increment; a change straddling a block boundary is caught by the next block's monitor, adding at most the current block's residual to the delay. Under the infinite-horizon alternative — prior weights $\frac{1}{r(r+1)}$ over restart times, summing to one — validity needs no horizon, but the mixture mass at restart τ is $\Theta(\tau^{-2})$, so the threshold term becomes $\log(\tau^2|W|)$ and the delay inherits a $\log \tau$ dependence; the block construction avoids this. $\square$

Remark 3 (Audit subsampling). Let each query be audited independently with probability r and let the monitor be run on the audited subsequence. Conditional on being audited, a query is an unbiased draw from the deployment distribution (the audit coin is independent of the query), so the audited losses $(\ell_{t_j})_j$ satisfy the same conditional-mean hypotheses as the full stream, and Propositions 3–4 apply verbatim with t counting audited queries. A post-change window of N audited queries occupies N/r wall-clock queries in expectation (and concentrates at rate $O(\sqrt{N}/r)$), giving the $1/r$ delay dilation quoted in Section 4.6.